

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника
направленность (профиль)
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка телеграм-бота для управления задачами в ЦБС Петроградского района

Обучающегося 4 курса
очной формы обучения
Суворова Романа Михайловича

Руководитель выпускной квалификационной
работы:
Доцент кафедры ИТиЭО
Жуков Николай Николаевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Глава 1. Аналитическая часть	6
1.1 Анализ существующих систем учёта заявок	6
1.2 Требования к разрабатываемому комплексу	8
1.3 Ролевая модель и жизненный цикл заявки	10
1.4 Выбор технологий и архитектурных решений	13
1.5 Проектирование архитектуры	17
Глава 2. Проектная часть	21
2.1 Реализация модели данных и схемы базы данных	21
2.2 Реализация серверного ядра helperbot-core	23
2.3 Реализация Telegram-клиента helperbot-tg	26
2.4 Аутентификация и разграничение прав доступа	29
2.5 Тестирование системы	32
2.6 Развёртывание и инфраструктура	33
ЗАКЛЮЧЕНИЕ	36
Список литературы	38
ПРИЛОЖЕНИЯ	40
Приложение А. Схема базы данных программного комплекса Helperbot	40
Приложение Б. Реализация сервисного слоя жизненного цикла заявки	43
Приложение В. Конфигурация docker-compose инфраструктуры	49
Приложение Г. Обработчик создания заявки в Telegram-боте	53
Приложение Д. Снимки экрана интерфейса Telegram-бота	58

ВВЕДЕНИЕ

Работа сотрудников библиотек включает не только основную деятельность, но и регулярное взаимодействие с инженерной службой: неисправности техники, обновление программ, проблемы с доступом к электронным ресурсам, замена расходных материалов. В Централизованной библиотечной системе Петроградского района Санкт-Петербурга (далее — ЦБС Петроградского района) подобные обращения передаются преимущественно неформальными средствами — через мессенджеры, телефонные звонки и устные просьбы. Такой порядок приводит к потере заявок, дублированию работы инженеров и отсутствию единой истории обращений.

Существующие системы учёта задач (Jira, Trello, ServiceDesk) для учреждений бюджетной сферы оказываются избыточными: они требуют оплаты лицензий и в большинстве случаев не допускают гибкой переработки под внутренние процессы организации. При этом сотрудники библиотек уже пользуются Telegram, что делает его удобной точкой входа в систему регистрации заявок без необходимости устанавливать дополнительное программное обеспечение и проходить отдельное обучение.

Актуальность работы обусловлена потребностью ЦБС Петроградского района в собственном средстве учёта технических заявок, доступном через привычный сотрудникам мессенджер и допускающем дальнейшее развитие — подключение новых клиентов (веб-интерфейс, мессенджер Max), интеграцию с внешними сервисами, в том числе с подсистемой рекомендаций литературы и средствами на основе искусственного интеллекта.

Объект исследования — процесс регистрации, распределения и сопровождения технических заявок сотрудников в библиотечной системе.

Предмет исследования — программные методы построения микросервисных приложений на языке Go с пользовательским интерфейсом в виде Telegram-бота для управления заявками.

Теоретическая значимость работы заключается в систематизации подходов к проектированию микросервисных приложений на языке Go, анализе взаимодействия независимых сервисов через gRPC и обосновании архитектурных решений, при которых единое серверное ядро допускает подключение нескольких клиентских приложений без изменения бизнес-логики.

Практическая значимость работы состоит в разработке программного комплекса *Helperbot*, предназначенного для применения в ЦБС Петроградского района. Комплекс позволяет: принимать заявки от библиотекарей через Telegram-бот; распределять заявки между инженерами с учётом приоритета; сопровождать заявку на всех этапах её жизненного цикла — от создания до закрытия; хранить историю обращений в реляционной базе данных; подключать к серверному ядру новые точки входа (веб-интерфейс, иные мессенджеры) без переработки доменной логики; взаимодействовать с внешним сервисом *helperbot-ai* для последующего расширения функциональности. Применяемый подход к разделению ядра и клиентских приложений может быть использован при проектировании аналогичных систем учёта обращений в других бюджетных учреждениях.

Цель выпускной квалификационной работы — разработать программный комплекс на основе Telegram-бота для учёта, распределения и сопровождения технических заявок сотрудников ЦБС Петроградского района.

В рамках достижения указанной цели сформулированы следующие **задачи**:

1. Изучить организацию работы сотрудников ЦБС и сформулировать функциональные и нефункциональные требования к системе учёта заявок.
2. Провести сравнительный анализ существующих средств управления задачами и обосновать выбор технологий для разработки.
3. Спроектировать архитектуру комплекса из двух независимых сервисов: серверного ядра *helperbot-core* и клиента *helperbot-tg*.
4. Спроектировать модель данных и подготовить миграции базы данных PostgreSQL для хранения сведений о пользователях, подразделениях и заявках.
5. Реализовать серверное ядро с gRPC-интерфейсом и REST-шлюзом, обеспечивающее жизненный цикл заявки.
6. Реализовать Telegram-бот с ролевой моделью (администратор, инженер, библиотекарь) и сценариями создания, назначения и закрытия заявок.
7. Подготовить инфраструктуру развёртывания на базе Docker Compose и провести тестирование разработанного комплекса.

Результатом бакалаврской выпускной квалификационной работы является программный комплекс *Helperbot* — система учёта технических заявок ЦБС Петроградского района, включающая серверное ядро с gRPC-интерфейсом, Telegram-бот с

ролевой моделью пользователей и инфраструктуру развёртывания. Комплекс разработан с расчётом на дальнейшее расширение за счёт подключения новых клиентских приложений и внешних сервисов.

Состав работы. Текст ВКР включает введение, две главы основной части, заключение, список литературы и приложения. В первой главе проводится анализ предметной области: рассматривается организация работы инженерной службы ЦБС, сравниваются существующие средства управления задачами, обосновывается выбор технологий и проектируется архитектура комплекса. Во второй главе описывается практическая реализация: модель данных, серверное ядро с gRPC-интерфейсом и REST-шлюзом, Telegram-бот с ролевой моделью пользователей, а также инфраструктура развёртывания.

ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 АНАЛИЗ СУЩЕСТВУЮЩИХ СИСТЕМ УЧЁТА ЗАЯВОК

Задачи поддержки и обработки внутренних обращений сотрудников решаются с помощью систем учёта заявок (*ticketing systems, issue trackers*). Такие системы позволяют фиксировать обращения, назначать ответственных исполнителей, отслеживать статус выполнения и хранить историю работ. В коммерческой сфере распространено несколько готовых решений, однако их применение в бюджетных учреждениях — в частности, в централизованных библиотечных системах — сопряжено с рядом ограничений. Прежде чем обосновать необходимость разработки собственного программного комплекса, рассмотрим наиболее распространённые готовые системы и сопоставим их возможности с требованиями к разрабатываемому комплексу *Helperbot* — системе учёта заявок с интерфейсом в виде Telegram-бота, рассчитанной на работу с собственной инфраструктурой ЦБС Петроградского района.

Jira Software — одна из самых популярных систем управления задачами и проектами, разработанная компанией Atlassian [1]. Платформа предоставляет гибкий механизм настройки рабочих процессов, поддерживает Scrum и Kanban доски, обладает развитой системой отчётности и интегрируется с большинством инструментов разработки. Несмотря на функциональность, Jira имеет ряд особенностей, затрудняющих её использование в небольших бюджетных учреждениях. Лицензия Jira Cloud распространяется по подписке, стоимость которой растёт пропорционально числу пользователей; ставку для облачной версии сложно согласовать в рамках государственного финансирования. После ухода Atlassian с российского рынка приобретение и продление лицензий на территории Российской Федерации усложнено. Помимо этого, интерфейс Jira ориентирован на ИТ-специалистов и требует обучения сотрудников, не имеющих опыта работы с трекерами задач.

Trello — облачный сервис управления задачами по методологии Kanban [2]. Интерфейс построен вокруг визуальных досок с карточками, что делает систему интуитивно понятной для новых пользователей. Однако Trello не предоставляет полноценных механизмов учёта заявок: отсутствует ролевая модель с разграничением прав на уровне отдельных задач, нет встроенного жизненного цикла обращения с фиксированными статусами, ограничены возможности отчётности. Бесплатный тариф

Trello накладывает лимит на количество досок и вложений, а коммерческие тарифы, как и в случае Jira, оплачиваются в иностранной валюте.

Redmine — проект с открытым исходным кодом, написанный на языке Ruby и распространяемый по лицензии GNU GPL v2 [3]. Redmine не требует лицензионных отчислений и допускает развёртывание на собственных серверах, что соответствует требованиям государственных учреждений к информационной безопасности. Тем не менее интерфейс Redmine ориентирован на разработчиков программного обеспечения: типичные пользователи библиотек, чья основная деятельность не связана с информационными технологиями, испытывают трудности при освоении системы. Для работы с Redmine сотрудникам потребуется отдельный браузерный клиент или мобильное приложение, что увеличивает порог входа.

Битрикс24 — российская платформа, объединяющая инструменты управления задачами, CRM и корпоративных коммуникаций [4]. Платформа имеет бесплатный тариф для небольших команд и поддерживает развёртывание на собственной инфраструктуре в варианте Битрикс24 в коробке. К её недостаткам относится избыточная для задачи учёта заявок функциональность: значительная часть возможностей платформы (CRM, сделки, телефония) не требуется для библиотечной системы и усложняет обучение сотрудников. Развёртывание коробочной версии предполагает работу со специализированным стеком (PHP, MySQL) и требует отдельного администратора.

ManageEngine ServiceDesk Plus — специализированная система управления обращениями в формате IT Service Desk [5]. Система предоставляет полный жизненный цикл заявки, поддерживает SLA, автоматическое распределение тикетов и интеграцию с базами активов. Однако ServiceDesk Plus распространяется по платной лицензии, ориентированной на корпоративные ИТ-подразделения, что делает её непрактичной для небольшой библиотечной системы.

Результаты сравнительного анализа представлены в таблице 1.1.

Из таблицы 1.1 видно, что ни одна из рассмотренных систем не отвечает одновременно всем условиям задачи. Коммерческие платформы (Jira, Trello, ServiceDesk Plus) предполагают платную подписку и в части случаев недоступны для покупки на территории Российской Федерации. Бесплатные решения (Redmine) либо требуют от пользователей навыков работы с веб-интерфейсом для разработчиков, либо рассчитаны на крупные организации (Битрикс24). Ни одна из систем не предоставляет родного Telegram-клиента, тогда как Telegram уже используется сотрудниками библиотек для рабочего общения. Совокупность перечисленных факторов делает

Таблица 1.1 – Сравнительный анализ существующих систем учёта заявок

Система	Открытый код	Бесплатна	Развёрт. on-premise	Telegram-клиент	Подходит для библиотек
Jira Software	Нет	Нет	Только для Data Center	Только сторонние плагины	Нет (платно, сложно для не-ИТ)
Trello	Нет	Лимиты	Нет	Сторонние боты	Нет (нет ролей, статусов)
Redmine	Да	Да	Да	Сторонние плагины	Частично (сложен для библиотек)
Битрикс24	Нет	Лимиты	Да (коробка)	Через интеграции	Частично (избыточная функциональность)
ServiceDesk Plus	Нет	Нет	Да	Сторонние плагины	Нет (дорого, для крупного ИТ)
Helperbot (разрабатываемый)	Да	Да	Да	Встроенный	Да

целесообразной разработку собственного программного комплекса, рассчитанного на специфику работы ЦБС Петроградского района.

1.2 ТРЕБОВАНИЯ К РАЗРАБАТЫВАЕМОМУ КОМПЛЕКСУ

На основе анализа предметной области и недостатков существующих систем были сформулированы требования к разрабатываемому программному комплексу *Helperbot*.

Функциональные требования:

1. Поддержка трёх ролей пользователей: библиотекарь, инженер, администратор; а также отдельной роли root-администратора, выполняющего первичную настройку системы.
2. Регистрация заявки библиотекарем с указанием описания проблемы, подразделения и приоритета (низкий, минорный, мажорный, критический).
3. Возможность создания заявки как по типовому шаблону из предопределённого списка, так и в свободной форме.
4. Просмотр администратором всех открытых заявок и назначение инженера-исполнителя.
5. Просмотр инженером закреплённых за ним заявок.
6. Просмотр библиотекарем созданных им заявок.

7. Изменение статуса заявки (создана, в работе, выполнена, удалена) с фиксацией времени перехода.
8. Аутентификация сотрудников по электронной почте и одноразовому паролю.
9. Привязка пользователя к подразделению ЦБС.
10. Хранение полной истории заявок с возможностью последующей выгрузки.
11. Возможность подключения к ядру внешних клиентов помимо Telegram-бота (веб-интерфейс, другие мессенджеры) без переработки бизнес-логики.
12. Возможность интеграции с внешним AI-сервисом для последующего расширения функциональности.

Нефункциональные требования:

1. Используемые технологии должны быть открытыми и не требовать лицензионных отчислений.
2. Время отклика Telegram-бота при выполнении типовой операции (создание заявки, просмотр списка) не должно превышать 2 секунд.
3. Архитектура должна допускать горизонтальное масштабирование серверного ядра и независимое развёртывание клиентских приложений.
4. Запуск всех компонентов комплекса должен производиться единым скриптом с помощью контейнеризации, без ручной настройки рантайма на целевой машине.
5. Заявки и связанные с ними данные должны сохраняться при перезапуске сервисов; история обращений не должна теряться при обновлении версий компонентов.
6. Отказ одного из клиентских приложений (например, Telegram-бота) не должен приводить к недоступности серверного ядра для других клиентов.

Перечисленные функциональные требования соответствуют сценариям взаимодействия трёх категорий пользователей с системой. Для наглядного представления полного перечня сценариев построена диаграмма прецедентов, представленная на рисунке 1.1. На диаграмме отражены варианты использования каждой роли и взаимодействие с центральным компонентом — серверным ядром, обрабатывающим бизнес-логику.

Helperbot — варианты использования (с разделением этапов)

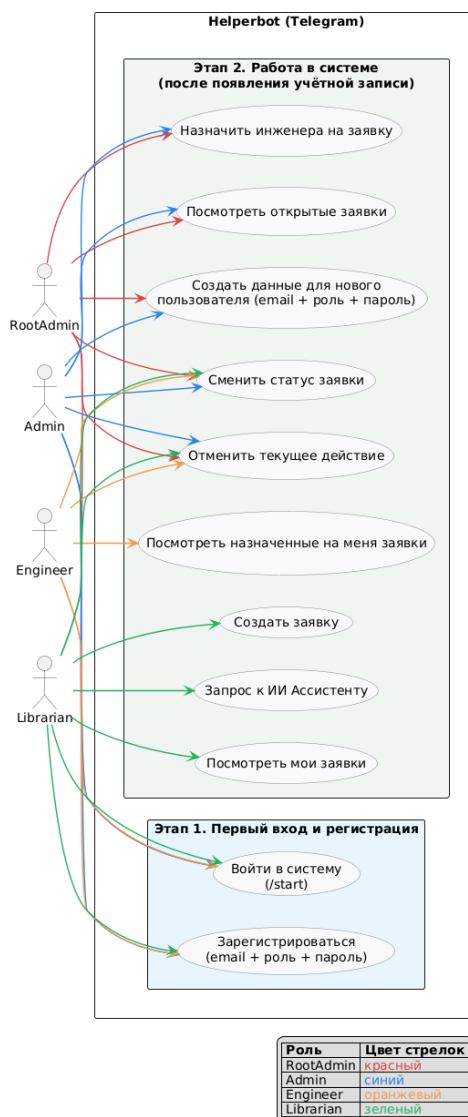


Рисунок 1.1 – Диаграмма прецедентов программного комплекса Helperbot

Как видно из рисунка 1.1, библиотекарь работает со сценариями создания и просмотра собственных заявок, инженер — с заявками, назначенными на него, администратор отвечает за распределение заявок между инженерами и общий контроль. Сценарий аутентификации общий для всех ролей и выполняется один раз при первом обращении пользователя к боту.

1.3 РОЛЕВАЯ МОДЕЛЬ И ЖИЗНЕННЫЙ ЦИКЛ ЗАЯВКИ

Поскольку основная задача комплекса — обработка обращений сотрудников, центральным понятием предметной области выступает **заявка**. Каждая заявка проходит несколько состояний, переход между которыми выполняется определёнными ролями пользователей. Корректная формализация состояний и переходов важна для

исключения ситуаций, при которых, например, библиотекарь пытается закрыть чужую заявку или инженер изменяет статус заявки, на которую не назначен.

Роли пользователей

В системе предусмотрены четыре роли, перечень которых приведён в таблице 1.2.

Таблица 1.2 – Роли пользователей программного комплекса Helperbot

Роль	Назначение и доступные действия
Библиотекарь	Создаёт заявки от имени своего подразделения, просматривает свои открытые заявки, может отменить созданную заявку.
Инженер	Получает назначенные на него заявки, изменяет их статус (в работе, выполнена).
Администратор	Просматривает все открытые заявки, назначает инженера-исполнителя, контролирует ход выполнения.
Root-администратор	Регистрирует учётные записи новых сотрудников, генерирует одноразовые пароли, обладает всеми правами администратора.

Разделение администратора и root-администратора обусловлено требованиями к разграничению прав доступа: операции, влияющие на учётные данные сотрудников, не должны быть доступны рядовому администратору. В коде ролевая модель оформлена в виде доменного перечисления UserRole, значения которого хранятся в базе данных в виде строки фиксированного набора (admin, engineer, librarian, undefined) [6].

Состояния заявки

Жизненный цикл заявки описывается набором из четырёх состояний и переходов между ними: **created** (заявка создана библиотекарем, исполнитель не назначен); **in_progress** (администратор назначил инженера, заявка взята в работу); **completed** (инженер отметил заявку как выполненную); **deleted** (заявка отменена автором либо удалена администратором). Диаграмма состояний приведена на рисунке 1.2.

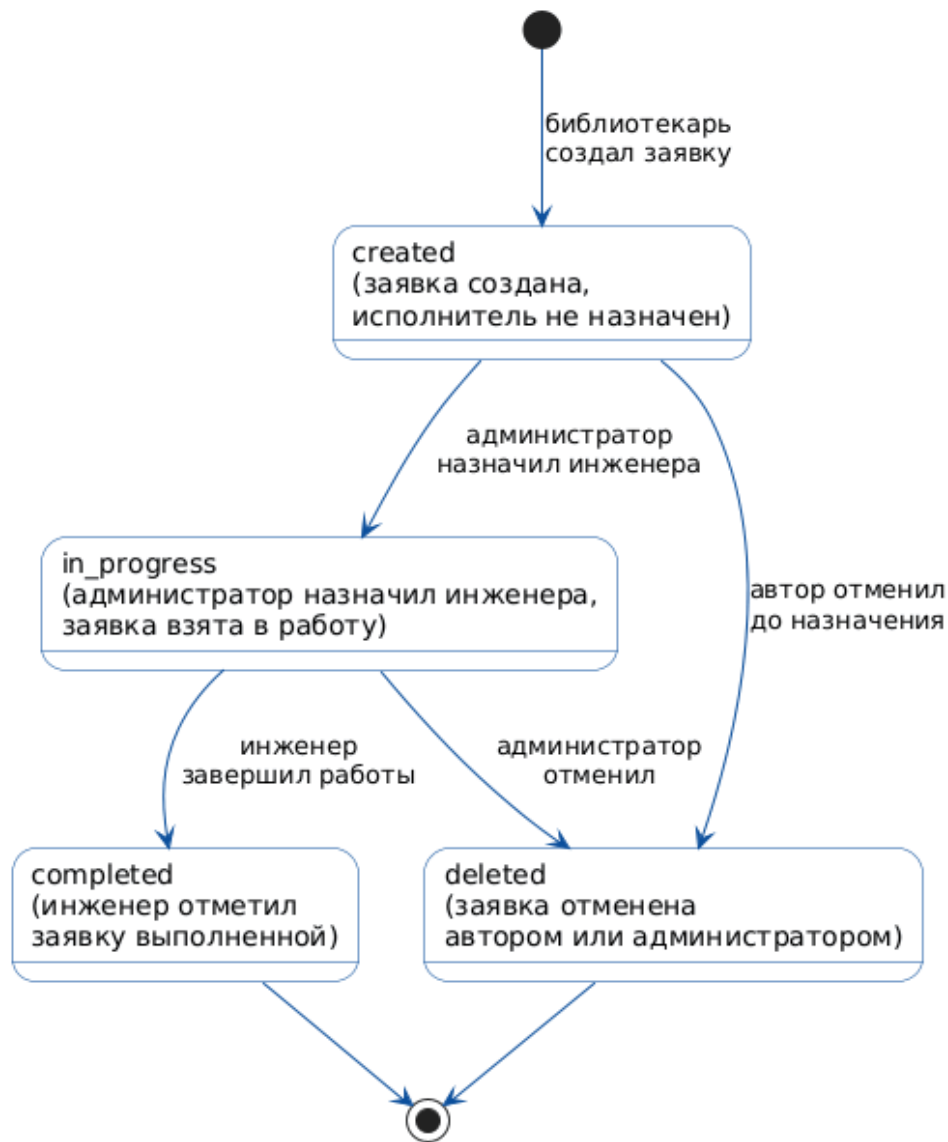


Рисунок 1.2 – Диаграмма состояний заявки в комплексе Helperbot

Согласно рисунку 1.2, переход из состояния **created** в **in_progress** происходит при назначении инженера администратором. Заявка в работе может быть либо успешно завершена инженером (переход в **completed**), либо удалена администратором или библиотекарем (переход в **deleted**). Из состояния **created** также возможен прямой переход в **deleted**, если библиотекарь отменяет заявку до того, как исполнитель был назначен. Терминальные состояния **completed** и **deleted** не допускают дальнейших переходов, что гарантирует невозможность повторного открытия закрытой заявки.

Право выполнять каждый из переходов закреплено за определённой ролью. Соответствие переходов и ролей приведено в таблице 1.3.

Формализованное описание переходов на уровне доменной модели позволяет вынести проверку допустимости переходов в один слой кода и не дублировать её в каждом обработчике Telegram-бота или вызове gRPC-метода. Реализация перехода

Таблица 1.3 – Допустимые переходы состояний заявки

Переход	Инициатор	Условие
created → in_progress	Администратор	Назначен инженер
created → deleted	Библиотекарь (автор), Администратор	Отмена до начала работ
in_progress → completed	Инженер-исполнитель	Работы завершены
in_progress → deleted	Администратор	Отмена выполняемой заявки

производится в слое сервисов и сопровождается обновлением полей `status`, `updated_at`, а для терминальных состояний — `closed_at` либо `deleted_at`.

1.4 ВЫБОР ТЕХНОЛОГИЙ И АРХИТЕКТУРНЫХ РЕШЕНИЙ

Состав технологий определялся исходя из сформулированных нефункциональных требований: открытость, отсутствие лицензионных отчислений, возможность развёртывания на собственной инфраструктуре и возможность последующего расширения комплекса новыми клиентскими приложениями. Ниже приводится обоснование выбора по каждому ключевому компоненту.

Язык программирования серверной части

В качестве языка серверного ядра и Telegram-клиента выбран **Go** [7; 8]. Программа на Go компилируется в один статически слинкованный исполняемый файл без внешних зависимостей, что упрощает упаковку в Docker-контейнер и развёртывание на сервере организации: итоговый образ не содержит интерпретатора или виртуальной машины, запуск выполняется без дополнительной настройки рантайма. Стандартная библиотека и средства конкурентности (горутины, каналы) делают разработку сетевых сервисов относительно простой, а кросс-компиляция позволяет собирать бинарные файлы под целевую платформу без её фактического наличия.

При этом выбор Go относится только к двум основным сервисам комплекса. Поскольку межсервисное взаимодействие построено на gRPC с описанием контрактов в .proto-файлах, дополнительные модули, которые могут понадобиться в дальнейшем (например, отдельная служба рекомендаций или интеграция с внешней информационной системой), допустимо реализовывать на любом языке, для которого существует генератор клиентского и серверного кода gRPC.

Межсервисное взаимодействие

Между серверным ядром *helperbot-core* и клиентскими приложениями (в первую очередь Telegram-ботом *helperbot-tg*) требуется надёжный механизм передачи данных. Рассматривались два варианта: классическое REST API с обменом данными в JSON и gRPC с обменом сообщениями в формате Protocol Buffers.

REST + JSON имеет низкий порог входа, поддерживается всеми языками и инструментами разработки. Однако JSON-сериализация не предоставляет встроенной валидации сообщений, формат контрактов определяется отдельно через OpenAPI, а накладные расходы на текстовую сериализацию выше, чем у бинарных форматов.

gRPC (gRPC-Go [9], gRPC [10]) использует формат Protocol Buffers [11], что обеспечивает компактное бинарное представление сообщений и строгую типизацию. Контракты сервисов описываются в .proto-файлах, по которым автоматически генерируется код клиента и сервера. gRPC поддерживает двунаправленную потоковую передачу, что может понадобиться при последующем расширении системы. Для одновременной поддержки REST-клиентов применяется grpc-gateway [12] — обратный прокси, который по тем же .proto-файлам генерирует HTTP-обработчики, транслирующие REST-запросы в gRPC-вызовы. Такой подход позволяет один раз описать контракт и предоставить как gRPC-, так и REST-доступ к одной и той же бизнес-логике.

Хранилище данных

В качестве системы управления базами данных выбрана **PostgreSQL** [6]. Это открытая бесплатная СУБД, что отвечает требованию к стеку. Из конкретных возможностей, использованных в работе, важны физическая репликация master-slave (нужна для отказоустойчивой инсталляции) и наличие актуального драйвера под язык Go.

Для подключения к базе данных из Go используется драйвер pgx [13]. Изменения схемы оформляются в виде SQL-миграций и применяются утилитой goose [14], которая поддерживает последовательное накатывание и откат миграций по версиям.

Telegram-клиент

Для написания Telegram-бота на Go существует две распространённые библиотеки: go-telegram-bot-api [15] и telebot [16]. Первая представляет собой тонкую обёртку над Telegram Bot API [17]: работа с обновлениями выполняется через явный цикл с самостоятельным разбором JSON-структур. telebot даёт более высокий уровень

абстракции: обработчики команд и нажатий регистрируются декларативно методом `Handle`, имеется поддержка `inline`-клавиатур и контекста сообщения. Для разрабатываемого комплекса выбрана `telebot` четвёртой версии как актуальная стабильная ветка библиотеки с активной поддержкой; декларативный стиль регистрации обработчиков снижает объём шаблонного кода в сравнении с самостоятельным разбором обновлений.

Управление зависимостями

Серверное ядро и Telegram-клиент построены на принципе разделения кода по слоям: репозиторий, сервис, обработчик. Для сборки графа зависимостей между этими слоями применяется `uber-go/fx` [18] — контейнер внедрения зависимостей для Go. Контейнер хранит набор конструкторов компонентов и самостоятельно создаёт экземпляры в нужном порядке, исходя из типов их аргументов. Использование такого подхода удобно при разработке: добавление нового компонента сводится к регистрации его конструктора, ручная передача ссылок между слоями не требуется. Это упрощает и подмену реализаций при тестировании.

Таблица 1.4 – Технологический стек комплекса Helperbot

Компонент	Технология	Обоснование выбора
Язык серверной части	Go	Статическая сборка, конкурентность, нет JVM
Межсервисный обмен	gRPC + Protocol Buffers	Типизированные контракты, кодогенерация
REST-фасад	grpc-gateway	Один контракт для gRPC и REST
СУБД	PostgreSQL	Репликация master-slave, драйвер под Go, открытый код
Драйвер БД	pgx	Нативный драйвер без CGO
Миграции	goose	Версионизируемые SQL-миграции
Telegram-библиотека	tucnak/telebot v4	Декларативная регистрация обработчиков
DI-фреймворк	uber-go/fx	Управление зависимостями, lifecycle hooks
Контейнеризация	Docker + Docker Compose	Воспроизводимое развёртывание

Развёртывание

Развёртывание комплекса организовано через Docker [19] и Docker Compose [20]. Каждый сервис собирается в отдельный контейнер, описание состава запускаемых компонентов вынесено в файл `docker-compose.yml`. Такой подход исключает разногласия между средами разработки и эксплуатации и не требует ручной настройки рантайма на сервере организации.

Итоговый технологический стек комплекса представлен в таблице 1.4.

1.5 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ

С учётом выбранных технологий была спроектирована архитектура комплекса из двух независимых сервисов, взаимодействующих через gRPC. Компонентная структура комплекса приведена на рисунке 1.3.

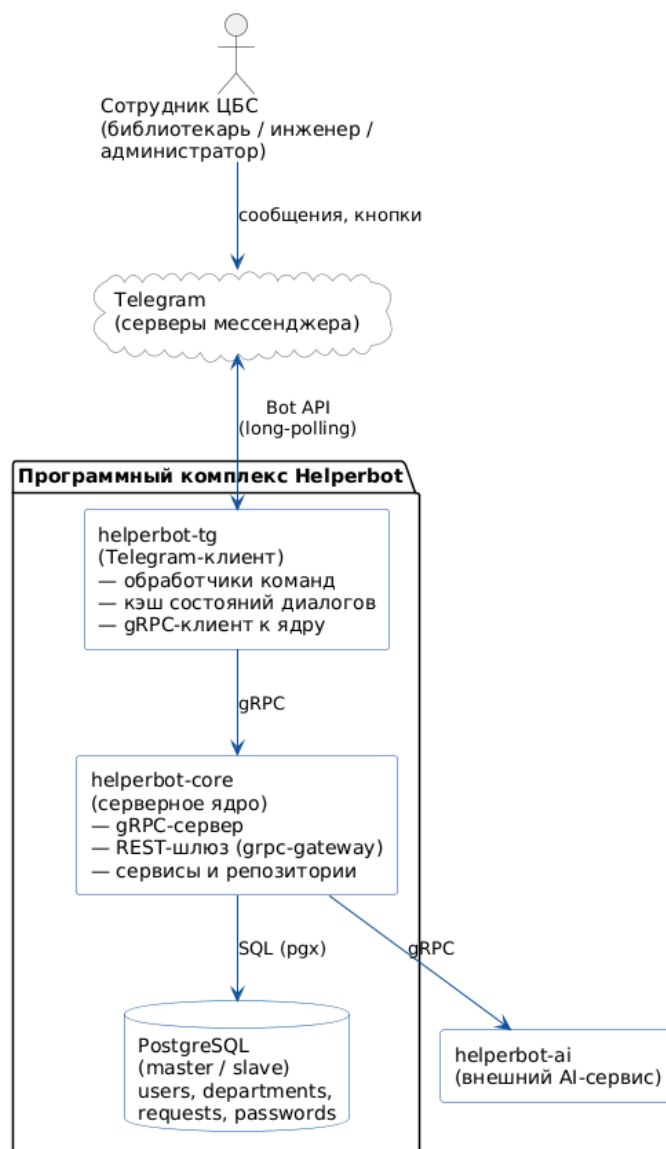


Рисунок 1.3 – Компонентная структура программного комплекса Helperbot

Серверное ядро helperbot-core

helperbot-core — центральный сервис комплекса, в котором сосредоточена вся бизнес-логика обработки заявок, пользователей и аутентификации. Внутренне сервис разбит на три слоя: **репозиторий** (работа с PostgreSQL, реализация интерфейсов доступа к данным), **сервис** (бизнес-логика, транзакции, проверка переходов жизненного цикла), **API** (gRPC-обработчики, входная точка для клиентов).

Помимо gRPC-сервера, ядро запускает HTTP-шлюз на основе grpc-gateway, который транслирует REST-запросы в gRPC-вызовы. Наличие шлюза даёт возможность в дальнейшем подключить к ядру веб-интерфейс или мобильное приложение без изменения серверного кода. Сборка приложения и регистрация всех зависимостей выполняются через библиотеку uber-go/fx, что позволяет описать состав сервиса в одном месте и заменять реализации компонентов при тестировании.

Клиентское приложение **helperbot-tg**

helperbot-tg — клиентское приложение, реализующее взаимодействие с пользователями через Telegram. Сервис содержит обработчики команд и сообщений на базе библиотеки telebot, кеш состояния диалогов пользователей и gRPC-клиентов, обращающихся к ядру.

Принципиально важным архитектурным решением является то, что *helperbot-tg* не работает напрямую с базой данных. Все обращения к доменным сущностям выполняются через gRPC-методы ядра, а Telegram-клиент отвечает только за презентационную логику: парсинг сообщений, формирование ответов, отображение клавиатур. Такое разделение даёт два преимущества: во-первых, поверх того же ядра можно подключить дополнительные клиенты (веб-страницу, мессенджер Max); во-вторых, изменения в бизнес-логике вносятся в одном месте.

Внешний сервис **helperbot-ai**

Помимо собственных сервисов, ядро интегрировано с внешним AI-сервисом *helperbot-ai*. Этот сервис разработан отдельной командой и в настоящей работе рассматривается только как внешняя зависимость с фиксированным gRPC-контрактом. Через данный канал планируется реализация дополнительных сценариев: получение справочных ответов, рекомендации литературы. В рамках архитектуры комплекса *helperbot-ai* подключается как gRPC-клиент: ядро поддерживает клиента ChatServiceClient и проксирует запросы пользователей.

База данных

В качестве хранилища применяется PostgreSQL. Схема данных содержит четыре основные таблицы: **departments** (подразделения ЦБС), **users** (сотрудники с привязкой

к подразделению и ролью), **requests** (заявки), **passwords** (одноразовые пароли для первичной аутентификации).

Для повышения отказоустойчивости в производственной инсталляции PostgreSQL развёрнута в режиме физической репликации master–slave: запись выполняется в основной экземпляр, чтение в перспективе может быть распределено между основным и репликой [21]. Итоговая схема базы данных приведена в приложении А.

Структура взаимодействия компонентов

Поток обработки типового сценария «библиотекарь создаёт заявку» выглядит следующим образом. Сообщение пользователя поступает в Telegram-бот по схеме long-polling — клиент бота отправляет к серверу Telegram длительный HTTP-запрос и ожидает на нём ответа до появления новых сообщений; после получения обновления запрос немедленно открывается заново. Такой механизм не требует наличия у бота публичного адреса и сертификата TLS, что необходимо при альтернативном подходе с использованием webhook. Обработчик *helperbot-tg* определяет состояние диалога, формирует объект заявки и вызывает gRPC-метод `RequestService.Create` в ядре. Ядро применяет бизнес-правила (проверка роли пользователя, установка статуса `created`), сохраняет заявку в PostgreSQL в рамках транзакции и возвращает созданную сущность. Telegram-клиент форматирует ответ и отправляет пользователю.

Подобная схема обеспечивает чёткое разделение ответственности: презентация и взаимодействие с пользователем сосредоточены в *helperbot-tg*, бизнес-логика и работа с данными — в *helperbot-core*. Принятая декомпозиция опирается на ряд принципов построения микросервисных систем, описанных в работе С. Ньюмана [22]. Во-первых, каждый сервис обладает собственной зоной ответственности и собственным набором интерфейсов: *helperbot-core* отвечает за доменную логику, *helperbot-tg* — за взаимодействие с конкретным каналом связи (Telegram). Во-вторых, прямой доступ к данным имеет только тот сервис, которому эти данные принадлежат: Telegram-клиент не подключается к базе напрямую, а работает только через gRPC-интерфейс ядра, что исключает неконтролируемые изменения схемы и упрощает поддержку. В-третьих, контракт взаимодействия между сервисами зафиксирован в .proto-файлах, что позволяет независимо развивать и развёртывать каждый компонент, сохраняя совместимость на уровне описания сообщений. Структура доменной модели (выделение сущностей *заявка*, *пользователь*, *подразделение*, инкапсуляция правил переходов состояний)

построена в соответствии с подходом предметно-ориентированного проектирования [23].

Таким образом, в первой главе проведён сравнительный анализ существующих систем учёта заявок, сформулированы функциональные и нефункциональные требования к комплексу, формализована ролевая модель и жизненный цикл заявки, обоснован выбор технологий и спроектирована архитектура из двух независимых сервисов с PostgreSQL в качестве хранилища. Принятые проектные решения реализованы во второй главе.

ГЛАВА 2. ПРОЕКТНАЯ ЧАСТЬ

2.1 РЕАЛИЗАЦИЯ МОДЕЛИ ДАННЫХ И СХЕМЫ БАЗЫ ДАННЫХ

В качестве хранилища данных применяется PostgreSQL. Схема описывает четыре доменные сущности: **подразделение** ЦБС Петроградского района, **сотрудник** (пользователь системы), **заявка** и **одноразовый пароль** для первичной аутентификации. Сущности связаны внешними ключами: пользователь привязан к подразделению, заявка ссылается на подразделение и на двух пользователей (библиотекаря-автора и инженера-исполнителя). Структура таблиц и связей приведена на рисунке 2.1.

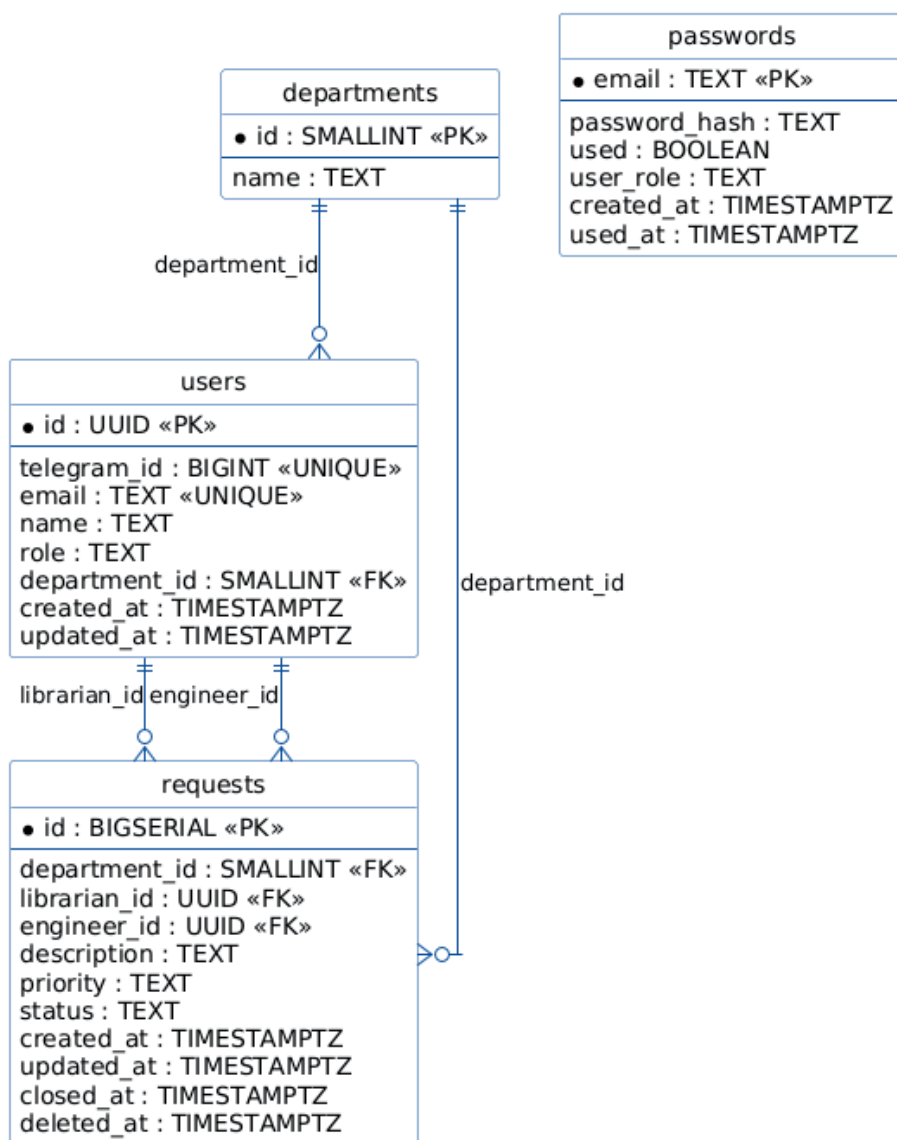


Рисунок 2.1 – Схема базы данных программного комплекса Helperbot

Назначение каждой таблицы и ключевые поля приведены в таблице 2.1.

Таблица 2.1 – Состав таблиц базы данных Helperbot

Таблица	Первичный ключ	Назначение и ключевые поля
departments	id SMALLINT	Справочник подразделений ЦБС; поле name
users	id UUID	Сотрудники; telegram_id, email, role, ссылка на departments
requests	id BIGSERIAL	Заявки; description, priority, status, ссылки на автора и исполнителя
passwords	email TEXT	Одноразовые пароли; password_hash, used, user_role

Поля `closed_at` и `deleted_at` в таблице `requests` имеют тип `NULLABLE TIMESTAMPTZ` и заполняются временной меткой при переходе заявки в соответствующее терминальное состояние. Идентификатор заявки реализован как `BIGSERIAL` — порядковый номер позволяет ссылаться на заявку в чате как на «заявку #123».

Доменные перечисления

Статусы заявки, её приоритеты и роли пользователей хранятся в виде текстовых значений (`TEXT`); допустимый набор значений фиксируется на уровне приложения. Текстовое представление делает содержимое таблиц читаемым без обращения к таблице соответствия числовых кодов и упрощает написание SQL-запросов вида `... WHERE status = 'in_progress'`. Полный перечень допустимых значений приведён в таблице 2.2.

Таблица 2.2 – Допустимые значения доменных перечислений

Перечисление	Значения
<code>user_role</code>	<code>undefined</code> , <code>admin</code> , <code>engineer</code> , <code>librarian</code>
<code>request_status</code>	<code>undefined</code> , <code>created</code> , <code>in_progress</code> , <code>completed</code> , <code>deleted</code>
<code>request_priority</code>	<code>undefined</code> , <code>low</code> , <code>minor</code> , <code>major</code> , <code>critical</code>

На уровне Go каждое перечисление оформлено отдельным типом (`UserRole`, `RequestStatus`, `RequestPriority`) с конструктором, валидацией и парой методов `String()/FromString()`. Такая обвязка гарантирует, что некорректное значение не попадёт в репозиторий из обработчика gRPC: ошибочная строка прерывается ещё на этапе разбора входящего сообщения.

Миграции

Управление схемой возложено на утилиту `goose` [14]. Каждая миграция оформлена как SQL-файл с секциями `+goose Up` и `+goose Down`, что обеспечивает применение и откат по версиям. Накатывание выполняется отдельным контейнером `db-migrations` в составе `docker-compose`: контейнер дожидается готовности мастера PostgreSQL, после чего вызывает `goose -dir /migrations postgres "..." up` и завершает работу. Итоговая схема базы данных (CREATE TABLE для четырёх таблиц с индексами и внешними ключами) приведена в приложении А.

2.2 РЕАЛИЗАЦИЯ СЕРВЕРНОГО ЯДРА HELPERBOT-CORE

Серверное ядро *helperbot-core* реализует доменную логику комплекса: создание, распределение и закрытие заявок, регистрацию учётных записей, проверку одноразовых паролей и проксирование запросов к внешнему AI-сервису. Сервис экспортирует две точки входа: gRPC-сервер для прикладных клиентов (в первую очередь *helperbot-tg*) и REST-шлюз для возможных будущих клиентов (веб-интерфейс, мобильное приложение).

Слоистая архитектура

Код ядра разделён на три слоя. **Слой репозитория** (`internal/repository`) работает напрямую со сторонними API и PostgreSQL через драйвер `pgx` [13]; каждая сущность имеет свой пакет с интерфейсом `Repository` и реализацией в подпакете `pg/`. **Слой сервисов** (`internal/service`) содержит бизнес-логику, не зависит от способа хранения и работает с репозиториями через интерфейсы. **Слой API** (`internal/api`) состоит из gRPC-обработчиков, которые валидируют входящие сообщения, преобразуют их в доменную модель и передают сервисам.

Зависимости направлены строго сверху вниз: API знает о сервисах, сервисы — о репозиториях, обратные ссылки исключены. Это допускает независимую замену реализации репозитория (например, на `in-memory` для отладки) без правок в сервисах и API.

Транзакционность

Операции, затрагивающие несколько таблиц (создание учётной записи с одноразовым паролем, регистрация сотрудника), оформлены в виде транзакций. Для передачи транзакции между слоями применяется приём «транзакция в контексте»: специальный сервис `transaction.Service` оборачивает функцию в `BEGIN/COMMIT/ROLLBACK`, а сам объект `pgx.Tx` помещает в `context.Context` через пакет `pkg/dbtx`. Репозитории на каждой операции вызывают `dbtx.ExtractTx(ctx)`: если транзакция в контексте есть — запрос выполняется через неё, иначе — через пул соединений.

Листинг 2.1: Метод `Service.Create` жизненного цикла заявки

```

1 func (s *Service) Create(
2     ctx context.Context, request model.Request,
3 ) (model.Request, error) {
4     request.Status = model.RequestStatusCreated
5
6     var createdRequest model.Request
7
8     if err := s.transactor.WithinTransaction(
9         ctx,
10        func(ctx context.Context) error {
11            var err error
12            createdRequest, err = s.requestRepository.Create(
13                ctx, request,
14            )
15            if err != nil {
16                return fmt.Errorf("create request: %w", err)
17            }
18            return nil
19        },
20    ); err != nil {
21        return model.Request{}, fmt.Errorf("with tx: %w", err)
22    }
23
24    return createdRequest, nil
25 }
```

Полный текст сервисного слоя жизненного цикла заявки (методы `Create`, `Update`, `Close` и `Delete`) приведён в приложении Б.

gRPC-API и REST-шлюз

Контракты сервисов описаны в .proto-файлах (директория api/); по ним кодогенератором protoc формируются клиентские и серверные заглушки на Go. Параметры запросов аннотированы правилами библиотеки protoc-gen-validate (например, (validate.rules).string.uuid = true для идентификаторов), что автоматически отсекает некорректные данные ещё до вызова бизнес-логики.

Поверх gRPC поднимается HTTP-шлюз на основе grpc-gateway [12]: HTTP-маршруты задаются в тех же .proto-файлах через аннотацию google.api.http, а сгенерированный обратный прокси транслирует REST-запросы в gRPC-вызовы. Это позволяет один раз описать контракт и предоставить клиентам выбор транспорта.

Листинг 2.2: Аннотация HTTP-маршрута в .proto-файле сервиса заявок

```

1 rpc Create(CreateRequest) returns (CreateResponse) {
2     option (google.api.http) = {
3         post: "/api/v1/request"
4         body: "*"
5     };
6 };

```

gRPC-сервер сконфигурирован с unary-интерсептором журналирования на базе slog и пакета go-grpc-middleware/v2: каждый завершённый вызов фиксируется в журнале с длительностью и кодом ответа. REST-шлюз дополнен CORS-обёрткой для разрешения запросов с веб-интерфейса на этапе разработки.

Управление зависимостями через uber-go/fx

Сборка приложения и регистрация компонентов выполняются через контейнер внедрения зависимостей uber-go/fx [18]. Каждый слой регистрирует свой конструктор в fx.Provide, а пары интерфейс-реализация — через fx.Annotate с fx.As. Запуск сетевых слушателей (gRPC и REST-шлюз) оформлен через хуки жизненного цикла fx.Hook: при старте приложения один хук открывает TCP-порт и запускает gRPC-сервер, другой — инициирует REST-шлюз. При получении сигнала остановки fx последовательно вызывает OnStop-хуки, в том числе закрывает пул соединений с базой данных.

2.3 РЕАЛИЗАЦИЯ TELEGRAM-КЛИЕНТА HELPERBOT-TG

Клиентское приложение *helperbot-tg* построено на библиотеке *telebot* четвёртой версии [16]. Его задача — принимать сообщения от сотрудников ЦБС, проводить их через многошаговые диалоги (например, «выбор подразделения → выбор приоритета → ввод описания») и обращаться к серверному ядру по gRPC за выполнением бизнес-логики.

Структура слоёв

Структура *helperbot-tg* в целом повторяет слоистую модель ядра: репозиторий, сервис, обработчик. Принципиальное отличие в том, что репозитории здесь не работают с базой данных, а оборачивают gRPC-клиенты к *helperbot-core*. Например, `request_backend.Repository` реализует тот же интерфейс `Repository`, что и в ядре, но вместо SQL-запроса вызывает `RequestServiceClient.Create`. За счёт этого сервисный слой клиента не знает, что он работает не с локальной базой, а с удалённым сервисом, и при появлении нового канала связи (веб, иной мессенджер) бизнес-логика клиента не меняется.

Кэш состояний пользовательских диалогов

Telegram-бот не поддерживает встроенного хранения состояния многошагового диалога с пользователем. Между двумя сообщениями требуется помнить, на каком шаге находится конкретный сотрудник и какие данные он уже ввёл. Для этого в клиенте реализован кэш `user_state.Cache` — потокобезопасное отображение `TelegramID` → `UserState` с TTL и фоновой очисткой по тикеру.

Структура `UserState` содержит четыре поля: текущее ожидаемое действие пользователя (`PendingAction`), частично заполненную учётную запись или заявку, список отправленных ботом сообщений (для последующего удаления при сбросе диалога) и идентификатор активного AI-чата, если он есть. По истечении TTL запись удаляется, а вместе с ней автоматически стираются вспомогательные сообщения бота, чтобы переписка не загромождалась оборванными диалогами.

Перечень ожидаемых действий приведён в таблице 2.3.

Таблица 2.3 – Шаги многошаговых диалогов (model.PendingAction)

Действие	Что ожидает бот
InputEmail	Email сотрудника при первичной регистрации
InputAdminEmail	Email нового пользователя (вводит администратор)
InputAdminRole	Роль для нового пользователя
InputPassword	Одноразовый пароль из чата с администратором
InputLibrarianRequestDescription	Свободное описание заявки от библиотекаря
GetAiMessage	Текст вопроса для AI-сервиса
WaitAiAnswer	Ожидание ответа AI; команда отмены ожидания

Диспетчеризация сообщений и кнопок

Точка входа — метод `Handler.handleText`, зарегистрированный на событие `telebot.OnText`. Сначала проверяется, есть ли у отправителя активный `PendingAction`: если есть, управление передаётся соответствующему обработчику (например, `processHandleEmail`). В противном случае текст сообщения сопоставляется с константами кнопок главного меню (`TextBtnCreateRequest`, `TextBtnAssignRequest` и т. д.) и запускается `start`-метод нужного сценария. Перед запуском сценария вызывается `checkPermission`, который читает учётную запись по `TelegramID` и сверяет роль пользователя со списком ролей, разрешённых для данной операции.

Inline-кнопки (выбор подразделения, приоритета, конкретной заявки из списка) регистрируются отдельно через `bot.Handle(&telebot.Btn{Unique: ...}, ...)`, что соответствует декларативной модели `telebot`. Идентификатор кнопки (`Unique`) служит ключом маршрутизации `callback`-сообщений к нужному методу обработчика.

Меню в зависимости от роли

Состав главного меню зависит от роли активного пользователя. Метод `Handler.showDefaultMenu` определяет роль (включая особый случай `root`-администратора — его `TelegramID` зафиксирован в конфигурации) и вызывает соответствующий метод построения меню. Например, библиотекарю предлагаются кнопки «Создать заявку», «Мои заявки», «Заккрыть заявку», «AI-чат»; инженеру — «Назначенные мне», «Заккрыть заявку», «AI-чат». Внешний вид меню библиотекаря приведён на рисунке 2.2. Совмещённое меню недопустимо: попытка библиотекаря инициировать сценарий назначения инженера блокируется на уровне `checkPermission`.

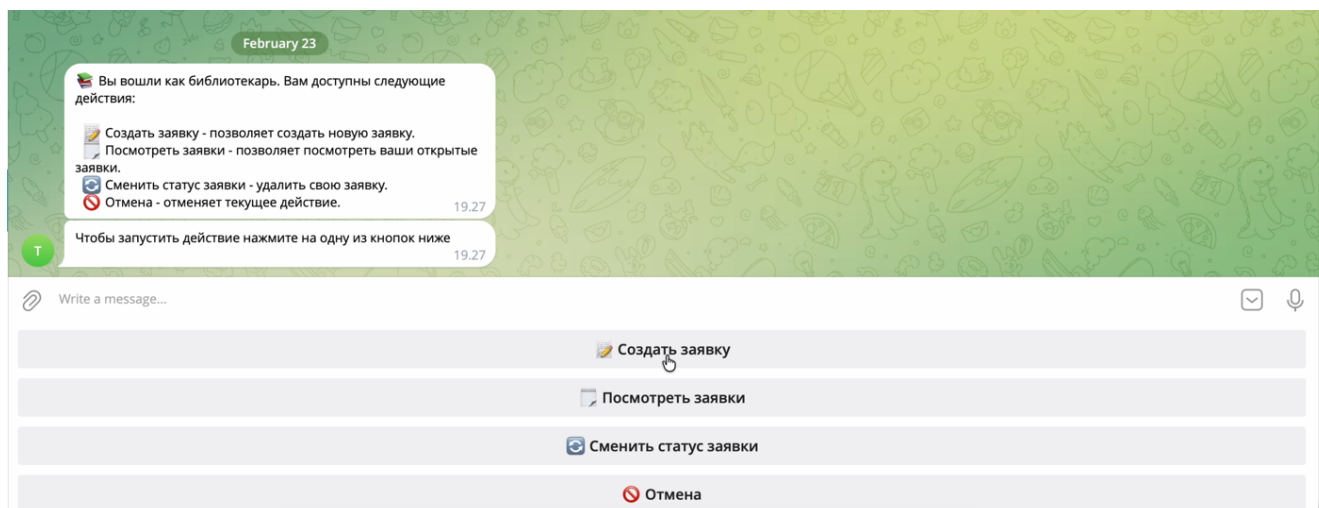


Рисунок 2.2 – Главное меню Telegram-бота для роли библиотекаря

Сценарий создания типовой заявки

После нажатия кнопки «Создать заявку» бот предлагает выбрать одну из заранее заданных формулировок типовых обращений или ввести собственное описание. Внешний вид inline-клавиатуры с типовыми формулировками приведён на рисунке 2.3.



Рисунок 2.3 – Выбор типовой заявки из списка predetermined формулировок

Полный путь от нажатия кнопки «Создать заявку» до записи в базе приведён на диаграмме последовательности на рисунке 2.4. Диаграмма охватывает шаги: по-

лучение сообщения от Telegram, диспетчеризация, выбор типового сценария через inline-меню, обращение к gRPC-методам *helperbot-core* и ответ пользователю.

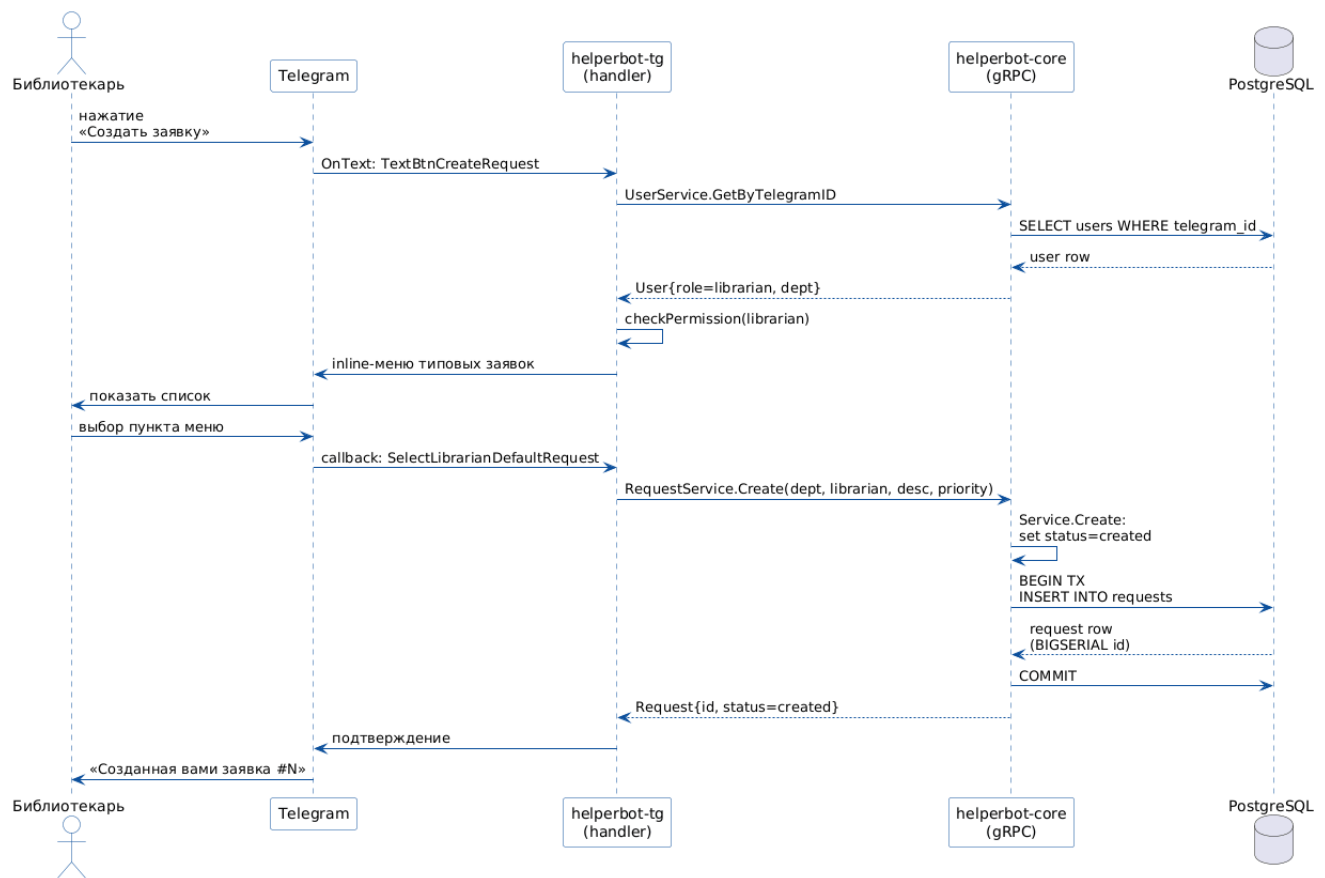


Рисунок 2.4 – Диаграмма последовательности создания типовой заявки

Метод-обработчик `processHandleLibrarianDefaultRequest` получает callback-сообщение Telegram с идентификатором выбранного шаблона, читает запись пользователя по `TelegramID`, проверяет роль и подразделение, после чего собирает доменную модель `Request` и вызывает `requestService.Create` — последняя обращается к серверному ядру по gRPC и возвращает созданную заявку. Полный текст обработчика приведён в приложении Г.

2.4 АУТЕНТИФИКАЦИЯ И РАЗГРАНИЧЕНИЕ ПРАВ ДОСТУПА

В отсутствие корпоративного каталога сотрудников и системы единого входа реализована схема аутентификации на основе одноразового пароля. Каждый сотрудник проходит её один раз: после успешной проверки пары «email, одноразовый пароль» к учётной записи привязывается `TelegramID` и в дальнейших сеансах сотрудник идентифицируется только по нему.

Регистрация сотрудника

Регистрация выполняется в два этапа разными лицами. На первом этапе администратор инициирует создание учётной записи в боте: вводит email будущего сотрудника, выбирает его роль и (для библиотекаря) подразделение. Серверное ядро через `LoginService.CreatePassword` генерирует случайный пароль (`uuid.New().String()`), хеширует его алгоритмом `bcrypt` и сохраняет запись в таблице `passwords` с привязкой к email и роли. Открытый пароль возвращается администратору сообщением в чате с ботом (рисунок 2.5) и передаётся сотруднику внутренними средствами организации (служебная записка, личная встреча, корпоративная переписка); через бота открытый пароль никогда не пересылается повторно.

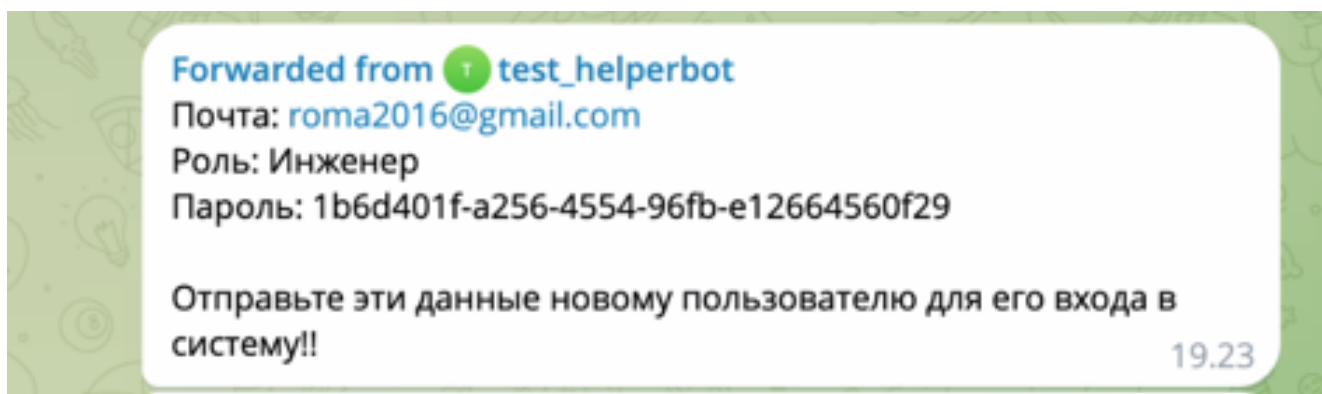


Рисунок 2.5 – Сообщение администратору с одноразовым паролем для нового сотрудника

На втором этапе сотрудник запускает бот и проходит диалог: команда `/start` переводит его в состояние `InputEmail`, бот спрашивает email, далее — одноразовый пароль. Метод `LoginService.CheckPassword` ядра выполняет проверку: достаёт хеш по email, убеждается, что пароль не был ранее использован, что роль совпадает с заявленной, сравнивает пароль с хешем (`bcrypt.CompareHashAndPassword`) и удаляет запись из таблицы `passwords`. После этого создаётся запись в таблице `users` с заполненным `telegram_id`, и сотрудник попадает в основное меню. Полный порядок действий обеих сторон представлен на диаграмме деятельности на рисунке 2.6.

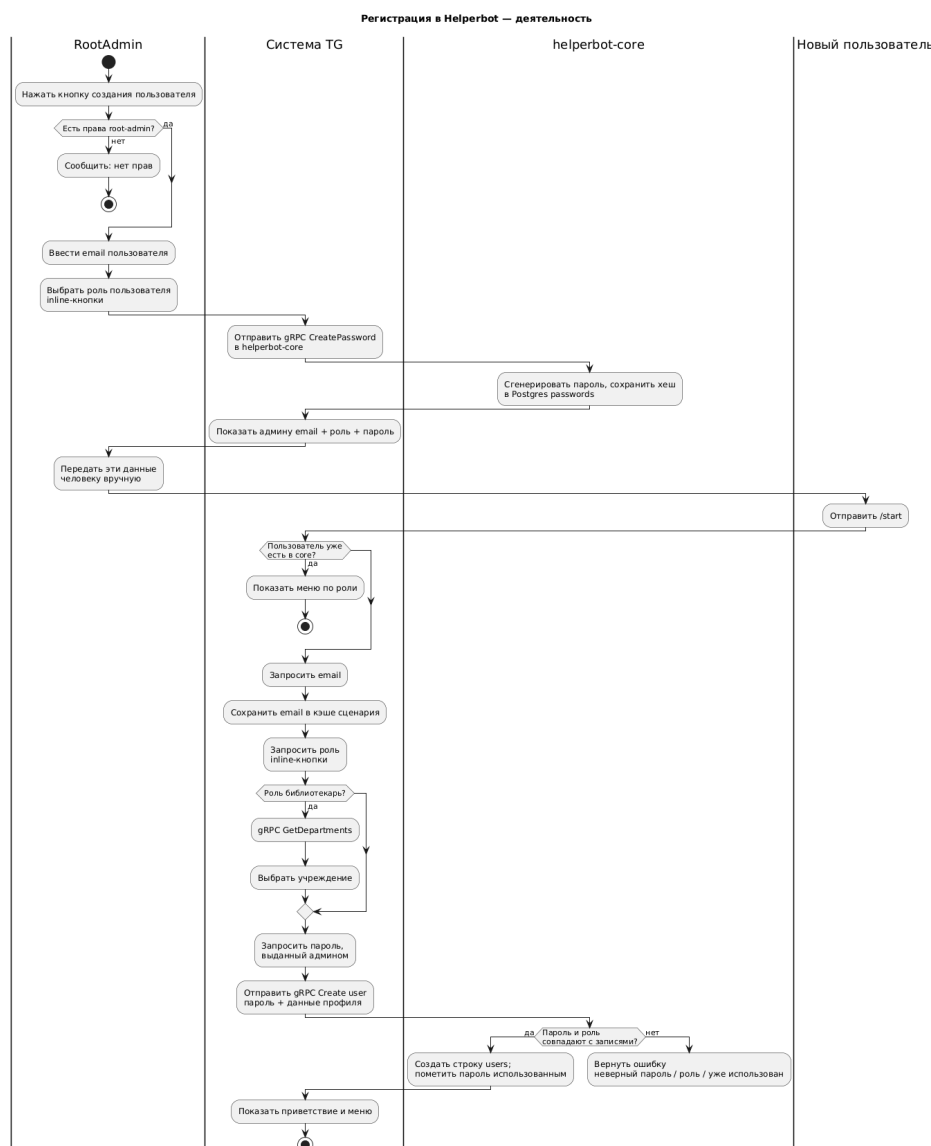


Рисунок 2.6 – Диаграмма деятельности регистрации сотрудника в комплексе Helperbot

Диаграмма разделена на четыре дорожки по участникам процесса: root-администратор инициирует создание учётной записи и передаёт сгенерированный пароль вручную; Telegram-бот *helperbot-tg* ведёт диалог как с администратором, так и с новым пользователем; ядро *helperbot-core* генерирует одноразовый пароль и впоследствии проверяет его; новый сотрудник проходит свою часть сценария от команды `/start` до получения основного меню.

Преимущества у схемы две. Во-первых, открытый пароль присутствует в системе только в момент передачи сотруднику и не хранится после первого использования. Во-вторых, сотрудник не вводит пароль при каждом обращении к боту: после первичной привязки идентификация выполняется на уровне Telegram.

Проверка прав на каждом шаге

Разграничение прав реализовано на уровне обработчика Telegram-бота. Каждый сценарий, доступный не всем ролям, начинается с вызова `checkPermission(ctx, s, allowedRoles...)`: метод получает учётную запись по TelegramID, сверяет роль со списком допустимых и, при несовпадении, отправляет пользователю сообщение об отказе и возвращает `ErrPermissionDenied`. Соответствие операций и ролей приведено в таблице 2.4.

Таблица 2.4 – Доступные операции по ролям пользователей

Операция	Допустимые роли
Создание учётной записи сотрудника	root-administrator, administrator
Создание заявки	librarian, administrator
Просмотр всех заявок	root-administrator, administrator
Просмотр своих заявок (как автор)	librarian, administrator
Просмотр назначенных заявок	engineer, administrator
Назначение инженера на заявку	root-administrator, administrator
Закрытие или отмена заявки	любая авторизованная роль (в рамках своих заявок)
Запуск AI-чата	любая авторизованная роль

Особый случай — root-администратор: его TelegramID прописан в конфигурационном файле клиента (`config.RootAdminId`) и проверяется в обход базы данных. Такая конструкция позволяет инициализировать систему «с нуля» — до того, как в таблице `users` появилась первая запись администратора.

2.5 ТЕСТИРОВАНИЕ СИСТЕМЫ

Для проверки корректности реализованного комплекса применено **ручное сценарное тестирование** [24]. Выбор такого подхода обусловлен двумя соображениями: число доменных сценариев ограничено и каждый из них имеет наблюдаемый эффект в интерфейсе Telegram-бота (новое сообщение, обновление списка заявок, переход в меню), а основная часть взаимодействия — последовательность реакций бота на сообщения пользователя — удобно проверяется человеком. Тестирование проводилось в локальной среде, развёрнутой через `docker-compose` с тестовой базой PostgreSQL; для каждой роли заводилась отдельная учётная запись через сценарий регистрации, после чего проигрывались целевые сценарии. Видеозапись прохождения ключевых сценариев за каждую из ролей доступна по ссылке [25].

Состав проверенных сценариев приведён в таблице 2.5.

Таблица 2.5 – Сценарии ручного тестирования программного комплекса Helperbot

Сценарий	Роль	Ожидаемый результат
Регистрация сотрудника	admin, librarian/engineer	Запись в users, удаление пароля из passwords
Повторное использование одноразового пароля	librarian	Отказ ErrPasswordUsed
Создание типовой заявки	librarian	Заявка со статусом created в requests
Создание заявки в свободной форме	librarian	Заявка с произвольным description
Просмотр своих заявок	librarian	Список содержит только созданные данным сотрудником заявки
Назначение инженера	admin	Статус заявки in_progress, engineer_id заполнен
Просмотр назначенных заявок	engineer	В списке только заявки, где engineer_id равен текущему
Закрытие заявки	engineer	Статус completed, closed_at заполнен
Отмена заявки до назначения	librarian	Статус deleted, deleted_at заполнен
Доступ к чужой операции	librarian	Сообщение об отказе, действие не выполняется
Перезапуск контейнеров	—	Список заявок и учётные записи сохраняются

По каждому сценарию проверялось не только сообщение бота, но и состояние базы данных: запросом SELECT убеждались, что соответствующая строка появилась, обновилась или изменила статус согласно ожиданиям. Сценарии разграничения прав проверялись попыткой выполнить недоступную операцию (например, библиотекарь нажимает кнопку «Назначить инженера»); во всех случаях бот возвращал сообщение об отказе, в журнале сервера фиксировалось предусмотренное ErrPermissionDenied.

Автоматизированных юнит- и интеграционных тестов в текущей версии комплекса не реализовано. Разработка покрытия тестами вынесена в дальнейшие задачи проекта: первоочередным кандидатом является сервисный слой *helperbot-core* (методы Create, Update, Close, Delete в пакете *service/request/v1*), где сосредоточены правила переходов жизненного цикла заявки.

2.6 РАЗВЁРТЫВАНИЕ И ИНФРАСТРУКТУРА

Развёртывание комплекса организовано так, чтобы инженеру ЦБС не требовалось вручную устанавливать Go, PostgreSQL или дополнительные утилиты: всё, что должно присутствовать на сервере организации, — это docker и docker-compose.

Состав docker-compose

Файл `docker-compose.yml` в репозитории *helperbot-infrastructure* описывает работу следующих контейнеров: двух экземпляров PostgreSQL в режиме физической репликации master-slave, контейнера однократных миграций db-migrations, сервиса *helperbot-core*, сервиса *helperbot-tg* и внешнего AI-сервиса *helperbot-ai* с отдельной парой PostgreSQL и моделью ollama. Назначение каждого контейнера описано в таблице 2.6.

Таблица 2.6 – Состав сервисов docker-compose комплекса Helperbot

Сервис	Назначение
postgresql-master	Основной экземпляр PostgreSQL, принимает запись
postgresql-slave	Реплика для отказоустойчивости и потенциального чтения
db-migrations	Однократно накатывает миграции через goose
helperbot-core	Серверное ядро (gRPC + REST-шлюз)
helperbot-tg	Telegram-клиент
helperbot-ai	Внешний AI-сервис, подключаемый по gRPC

Сервисы объединены в bridge-сеть *helperbot-network*; наружу публикуются только порты gRPC, REST-шлюза и (при необходимости) PostgreSQL для административного доступа. Конфигурация каждого сервиса передаётся через монтируемый `.env`-файл (`.env.helperbot-core`, `.env.helperbot-tg`, `.env.helperbot-ai`), что позволяет хранить параметры подключения и токены отдельно от исходного кода.

Сборка контейнеров

Образы *helperbot-core* и *helperbot-tg* собираются по многоступенчатой схеме: на этапе сборки используется образ `golang:alpine`, в котором компилируется бинарный файл, после чего исполняемый файл копируется в минимальный рантайм-образ `alpine:latest`. Финальный образ содержит только бинарный файл и стандартные библиотеки Alpine; среды выполнения языка, исходного кода и сборочных артефактов в нём нет. Такой подход уменьшает размер образа в несколько раз и сокращает поверхность атаки.

Миграции при первом запуске

Контейнер db-migrations использует базовый образ `golang:alpine`, в который устанавливается инструмент `goose` [14]; контейнер монтирует каталог migrations из репо-

зитория *helperbot-core* и применяет миграции командой `goose -dir /migrations postgres "<DSN>" up`. Зависимость `depends_on` с условием `service_healthy` master-экземпляра гарантирует, что миграции запускаются только после успешного `pg_isready`. После накатывания контейнер завершает работу с политикой `restart: "no"`; повторный запуск `compose up` выполнит только новые миграции. Полный текст `docker-compose.yml` приведён в приложении В.

Управление инфраструктурой через Makefile

Поверх `docker-compose` реализован `Makefile` в репозитории *helperbot-infrastructure*. Он скрывает работу с тремя отдельными репозиториями (*helperbot-core*, *helperbot-tg*, *helperbot-ai*) и предоставляет команды, описанные в таблице 2.7.

Таблица 2.7 – Команды `Makefile` репозитория *helperbot-infrastructure*

Команда	Действие
<code>make compose-up</code>	Подтянуть репозитории, собрать образы, запустить
<code>make compose-start</code>	Запустить уже собранные контейнеры
<code>make compose-stop</code>	Остановить контейнеры без удаления
<code>make update-repo</code>	Обновить исходные репозитории <i>helperbot-core</i> и <i>helperbot-tg</i>
<code>make migrate-up-core</code>	Накатить миграции <i>helperbot-core</i>
<code>make migrate-down-core</code>	Откатить последнюю миграцию <i>helperbot-core</i>

Сочетание `Makefile` и `docker-compose` позволяет развернуть рабочую копию комплекса на новой машине двумя действиями: подготовить три `.env`-файла по приведённым в README шаблонам и выполнить `make compose-up`.

Таким образом, во второй главе реализован программный комплекс *Helperbot*: спроектирована модель данных и применены миграции PostgreSQL, разработано серверное ядро *helperbot-core* со слоистой архитектурой и gRPC-интерфейсом, написан Telegram-клиент *helperbot-tg* с многошаговыми диалогами и ролевыми меню, реализована схема аутентификации по одноразовому паролю, проверены целевые сценарии ручным тестированием и подготовлена инфраструктура развёртывания на базе `docker-compose`.

ЗАКЛЮЧЕНИЕ

Итог работы — программный комплекс *Helperbot*: Telegram-бот, через который сотрудники ЦБС Петроградского района подают, отслеживают и закрывают заявки в инженерную службу. Цель, поставленная во введении, достигнута.

Ниже — короткий отчёт по каждой из задач, сформулированных во введении.

Проведён сравнительный анализ существующих средств управления задачами: Jira Software, Trello, Redmine, Битрикс24 и ManageEngine ServiceDesk Plus. В ходе анализа установлено, что ни одно из рассмотренных решений не сочетает одновременно открытость и бесплатность, возможность развёртывания на собственной инфраструктуре и встроенного клиента на базе мессенджера, которым уже пользуются сотрудники библиотечной системы. Это обосновало целесообразность разработки собственного комплекса.

Сформулированы функциональные и нефункциональные требования к разрабатываемому программному комплексу. Среди ключевых функциональных требований — поддержка ролей библиотекаря, инженера и администратора, полный жизненный цикл заявки от создания до закрытия, аутентификация по электронной почте и одноразовому паролю, возможность подключения к серверному ядру новых клиентских приложений. Среди нефункциональных требований — открытость технологического стека, горизонтальное масштабирование ядра, сохранность данных при перезапуске сервисов и независимость клиентов друг от друга.

Спроектирована архитектура комплекса из двух независимых сервисов, взаимодействующих через gRPC: серверного ядра *helperbot-core* со слоистой структурой «репозиторий–сервис–API» и клиентского приложения *helperbot-tg* на базе библиотеки *telebot*. Формализована модель данных и жизненный цикл заявки, описаны роли пользователей и допустимые переходы между состояниями.

Реализован программный комплекс. Разработана схема базы данных PostgreSQL из четырёх таблиц с применением миграций через *goose*; перечисления статусов, приоритетов и ролей хранятся в виде текстовых значений. Серверное ядро реализовано на Go: репозитории на драйвере *pgx* с поддержкой транзакций через контекст, бизнес-логика с проверкой правил переходов жизненного цикла, gRPC-интерфейс с REST-шлюзом *grpc-gateway*, сборка приложения через контейнер внедрения зависимостей

uber-go/fx. Telegram-клиент реализован с поддержкой многошаговых диалогов на основе кэша состояний с TTL и динамическими меню в зависимости от роли пользователя; за выполнением бизнес-операций клиент обращается к ядру по gRPC и не работает с базой данных напрямую. Реализована схема аутентификации по одноразовому паролю: открытый пароль существует в системе только до первого использования, после привязки TelegramID к учётной записи дальнейшая идентификация выполняется без повторного ввода учётных данных.

Подготовлена инфраструктура развёртывания на базе docker-compose: кластер PostgreSQL в режиме физической репликации master-slave, однократный контейнер миграций, серверное ядро и Telegram-клиент в виде многоступенчатых образов на основе alpine:latest. Поверх docker-compose реализован Makefile, позволяющий развернуть рабочую копию комплекса двумя действиями: подготовить три .env-файла по приведённым шаблонам и выполнить make compose-up.

Проведено ручное сценарное тестирование разработанного комплекса. Проверены ключевые сценарии каждой роли: регистрация сотрудника, создание заявки в типовом и свободном виде, назначение инженера, закрытие и отмена заявки, разграничение прав, сохранность данных при перезапуске контейнеров. По каждому сценарию контролировалось как поведение бота, так и состояние базы данных.

Практическая ценность разработанного комплекса состоит в том, что он закрывает потребность ЦБС Петроградского района в учёте технических заявок без необходимости приобретения коммерческих лицензий и обучения сотрудников работе с трекерами задач: интерфейс комплекса — привычный сотрудникам Telegram-бот, бизнес-логика хранится на инфраструктуре организации. Принятый подход к разделению ядра и клиентских приложений может быть повторён для аналогичных задач в других бюджетных учреждениях.

Перспективными направлениями развития комплекса являются: разработка веб-интерфейса для администратора через имеющийся REST-шлюз, покрытие сервисного слоя серверного ядра автоматическими тестами, расширение интеграции с внешним AI-сервисом для предоставления сотрудникам справочных ответов и рекомендаций литературы, а также подключение дополнительных клиентов на иных мессенджерах (в первую очередь Max).

СПИСОК ЛИТЕРАТУРЫ

1. Jira Software — Issue & Project Tracking Software. — 2026. — URL: <https://www.atlassian.com/software/jira> (дата обращения: 15.05.2026).
2. Trello — Manage Your Team’s Projects. — 2026. — URL: <https://trello.com> (дата обращения: 15.05.2026).
3. Redmine — flexible project management web application. — 2026. — URL: <https://www.redmine.org> (дата обращения: 15.05.2026).
4. Битрикс24 — корпоративный портал и CRM. — 2026. — URL: <https://www.bitrix24.ru> (дата обращения: 15.05.2026).
5. ManageEngine ServiceDesk Plus — Help Desk Software. — 2026. — URL: <https://www.manageengine.com/products/service-desk/> (дата обращения: 15.05.2026).
6. PostgreSQL: The World’s Most Advanced Open Source Relational Database. — 2026. — URL: <https://www.postgresql.org> (дата обращения: 15.05.2026).
7. The Go Programming Language. — 2026. — URL: <https://go.dev> (дата обращения: 15.05.2026).
8. The Go Programming Language Specification. — 2026. — URL: <https://go.dev/ref/спес> (дата обращения: 15.05.2026).
9. gRPC-Go: The Go implementation of gRPC. — 2026. — URL: <https://github.com/grpc/grpc-go> (дата обращения: 15.05.2026).
10. gRPC — A high performance, open source universal RPC framework. — 2026. — URL: <https://grpc.io> (дата обращения: 15.05.2026).
11. Protocol Buffers. — 2026. — URL: <https://protobuf.dev> (дата обращения: 15.05.2026).
12. grpc-gateway — generates a reverse-proxy server which translates a RESTful HTTP API into gRPC. — 2026. — URL: <https://github.com/grpc-ecosystem/grpc-gateway> (дата обращения: 15.05.2026).
13. pgx — PostgreSQL driver and toolkit for Go. — 2026. — URL: <https://github.com/jackc/pgx> (дата обращения: 15.05.2026).

14. *goose* — A database migration tool. Supports SQL migrations and Go functions. — 2026. — URL: <https://github.com/pressly/goose> (дата обращения: 15.05.2026).
15. Golang bindings for the Telegram Bot API. — 2026. — URL: <https://github.com/go-telegram-bot-api/telegram-bot-api> (дата обращения: 15.05.2026).
16. *telebot* — Telegram bot framework written in Go. — 2026. — URL: <https://github.com/tucnak/telebot> (дата обращения: 15.05.2026).
17. Telegram Bot API documentation. — 2026. — URL: <https://core.telegram.org/bots/api> (дата обращения: 15.05.2026).
18. Uber Fx — A dependency injection based application framework for Go. — 2026. — URL: <https://github.com/uber-go/fx> (дата обращения: 15.05.2026).
19. Docker — Accelerated Container Application Development. — 2026. — URL: <https://www.docker.com> (дата обращения: 15.05.2026).
20. Docker Compose documentation. — 2026. — URL: <https://docs.docker.com/compose/> (дата обращения: 15.05.2026).
21. *Клеппман М.* Высоконагруженные приложения. Программирование, масштабирование, поддержка. — СПб. : Питер, 2018. — 640 с. — ISBN 978-5-4461-0512-0.
22. *Ньюман С.* Создание микросервисов. — 2-е изд. — СПб. : Питер, 2021. — 624 с. — ISBN 978-5-4461-1834-2.
23. *Эванс Э.* Предметно-ориентированное проектирование (DDD). Структуризация сложных программных систем. — М. : Вильямс, 2011. — 448 с. — ISBN 978-5-8459-1597-9.
24. *Куликов С. С.* Тестирование программного обеспечения. Базовый курс. — 3-е изд. — Минск : Четыре четверти, 2020. — 312 с. — ISBN 978-985-581-362-1.
25. *Суворов Р. М.* Демонстрация работы программного комплекса Helperbot за каждую из ролей пользователей. — 2026. — URL: <https://drive.google.com/file/d/1YKiNXUpwlfKMV5EYnQqkLiQvcIR7K0mv/view?usp=sharing> (дата обращения: 25.05.2026).

ПРИЛОЖЕНИЯ

Приложение А. Схема базы данных программного комплекса Helperbot

В приложении представлена итоговая схема базы данных программного комплекса *Helperbot*, отражающая результат применения всех миграций. Схема включает четыре таблицы — *departments*, *users*, *passwords* и *requests* — а также внешние ключи между ними. Полные тексты миграций находятся в каталоге *migrations/* исходного кода сервиса *helperbot-core*.

Листинг 2.3: Итоговая схема базы данных Helperbot

```

1 CREATE TABLE departments (
2     id          SMALLINT PRIMARY KEY,
3     name        TEXT NOT NULL,
4     created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
5     updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
6     CONSTRAINT departments_id_uint8 CHECK (id BETWEEN 0 AND 255)
7 );
8
9 INSERT INTO departments (id, name) VALUES
10     (1, 'ЦРБ им. Пушкина'),
11     (2, 'ЦРДБ'),
12     (3, 'Библиотека им. В. И. Ленина'),
13     (4, 'Библиотека на Карповке'),
14     (5, 'Библиотека им. А. П. Гайдара'),
15     (6, 'Библиотека Кировских островов'),
16     (7, 'БКГ'),
17     (8, 'я2- детская'),
18     (9, 'Отделы на Каменноостровском 34'),
19     (10, 'ЦБС');
20
21 CREATE TABLE passwords (
22     email        TEXT PRIMARY KEY,
23     password_hash TEXT NOT NULL,
24     used         BOOLEAN NOT NULL DEFAULT FALSE,
25     user_role     TEXT NOT NULL DEFAULT 'undefined',

```



```

26     created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
27     used_at         TIMESTAMPTZ
28 );
29
30 CREATE TABLE users (
31     id              UUID PRIMARY KEY,
32     telegram_id     BIGINT UNIQUE NOT NULL,
33     name            TEXT NOT NULL,
34     email           TEXT UNIQUE NOT NULL,
35     role            TEXT NOT NULL DEFAULT 'undefined',
36     department_id   SMALLINT,
37     created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
38     updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
39     CONSTRAINT fk_users_department FOREIGN KEY (department_id)
40         REFERENCES departments(id),
41     CONSTRAINT fk_users_passwords_email FOREIGN KEY (email)
42         REFERENCES passwords(email)
43 );
44
45 CREATE TABLE requests (
46     id              BIGSERIAL PRIMARY KEY,
47     department_id   SMALLINT NOT NULL,
48     librarian_id    UUID NOT NULL,
49     engineer_id     UUID,
50     description     TEXT NOT NULL,
51     priority        TEXT NOT NULL DEFAULT 'undefined',
52     status          TEXT NOT NULL DEFAULT 'undefined',
53     created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
54     updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
55     closed_at       TIMESTAMPTZ,
56     deleted_at      TIMESTAMPTZ,
57     FOREIGN KEY (department_id) REFERENCES departments(id),
58     FOREIGN KEY (librarian_id) REFERENCES users(id),
59     FOREIGN KEY (engineer_id) REFERENCES users(id)
60 );
61
62 CREATE INDEX idx_requests_department_id ON requests(department_id);
63 CREATE INDEX idx_requests_librarian_id ON requests(librarian_id);
64 CREATE INDEX idx_requests_engineer_id ON requests(engineer_id);
65 CREATE INDEX idx_requests_priority ON requests(priority);
66 CREATE INDEX idx_requests_status ON requests(status);

```

```
67 CREATE INDEX idx_requests_created_at ON requests(created_at);  
68 CREATE INDEX idx_requests_updated_at ON requests(updated_at);
```

Значения колонок `role`, `priority` и `status` проверяются на уровне приложения через доменные перечисления (`UserRole`, `RequestPriority`, `RequestStatus`). Допустимые значения: для роли — `undefined`, `admin`, `engineer`, `librarian`; для приоритета — `undefined`, `low`, `minor`, `major`, `critical`; для статуса — `undefined`, `created`, `in_progress`, `completed`, `deleted`.

Приложение Б. Реализация сервисного слоя жизненного цикла заявки

В приложении приведён фрагмент исходного кода серверного ядра *helperbot-core*, реализующий сервисный слой обработки заявок. Файлы расположены в каталоге `internal/service/request/v1/` и отвечают за бизнес-логику жизненного цикла заявки: создание, изменение статуса, назначение исполнителя, закрытие и удаление. Каждая операция выполняется в рамках транзакции (`transactor.WithinTransaction`), что гарантирует консистентность данных при последовательных обращениях к репозиторию.

Листинг 2.4: Конструктор сервиса заявок (`service.go`)

```

1 package v1
2
3 import (
4     "github.com/cbs-helperbot/helperbot-core/internal/repository/
    request"
5     "github.com/cbs-helperbot/helperbot-core/internal/repository/user"
6     "github.com/cbs-helperbot/helperbot-core/internal/service/
    transaction"
7 )
8
9 type Service struct {
10     userRepository    user.Repository
11     requestRepository request.Repository
12     transactor         transaction.Service
13 }
14
15 func NewService(
16     userRepository user.Repository,
17     requestRepository request.Repository,
18     transactor transaction.Service,
19 ) *Service {
20     return &Service{
21         userRepository:    userRepository,
22         requestRepository: requestRepository,
23         transactor:         transactor,
24     }
25 }
```

Листинг 2.5: Создание новой заявки (`create.go`)

```

1 package v1
2
3 import (
4     "context"
5     "fmt"
6
7     "github.com/cbs-helperbot/helperbot-core/internal/model"
8 )
9
10 func (s *Service) Create(
11     ctx context.Context, request model.Request,
12 ) (model.Request, error) {
13     request.Status = model.RequestStatusCreated
14
15     var createdRequest model.Request
16
17     if err := s.transactor.WithinTransaction(ctx,
18         func(ctx context.Context) error {
19             var err error
20             createdRequest, err = s.requestRepository.Create(ctx,
21 request)
22             if err != nil {
23                 return fmt.Errorf("create request: %w", err)
24             }
25             return nil
26         }); err != nil {
27         return model.Request{}, fmt.Errorf("with tx: %w", err)
28     }
29
30     return createdRequest, nil
31 }

```

Листинг 2.6: Обновление заявки и проверка перехода статуса (update.go)

```

1 package v1
2
3 import (
4     "context"
5     "fmt"
6     "reflect"
7
8     "github.com/cbs-helperbot/helperbot-core/internal/model"

```

```

9 )
10
11 func (s *Service) Update(
12     ctx context.Context, request model.Request,
13 ) (model.Request, error) {
14     var updatedRequest model.Request
15
16     if err := s.transactor.WithinTransaction(ctx,
17         func(ctx context.Context) error {
18             storedRequest, err := s.requestRepository.Get(ctx, request.
ID)
19
20             if err != nil {
21                 return fmt.Errorf("get stored request: %w", err)
22             }
23             if reflect.DeepEqual(storedRequest, model.Request{}) {
24                 return model.ErrRequestNotFound
25             }
26
27             engineerID, ok := request.EngineerID.Get()
28             if ok {
29                 engineer, err := s.userRepository.Get(ctx, engineerID)
30                 if err != nil {
31                     return fmt.Errorf("get engineer: %w", err)
32                 }
33                 if reflect.DeepEqual(engineer, model.User{}) {
34                     return model.ErrUserNotFound
35                 }
36                 if engineer.Role != model.UserRoleEngineer {
37                     return fmt.Errorf("user is not an engineer")
38                 }
39                 storedRequest.EngineerID = request.EngineerID
40             }
41
42             if request.Description != "" {
43                 storedRequest.Description = request.Description
44             }
45             if request.Priority != 0 {
46                 storedRequest.Priority = request.Priority
47             }
48
49             if request.Status != model.RequestStatusInProgress {

```

```

49         return model.ErrWrongRequestStatus
50     }
51     if storedRequest.Status == model.RequestStatusCreated ||
52        storedRequest.Status == model.RequestStatusInProgress {
53        storedRequest.Status = request.Status
54    } else {
55        return model.ErrWrongRequestStatus
56    }
57
58    updatedRequest, err = s.requestRepository.Update(ctx,
storedRequest)
59    if err != nil {
60        return fmt.Errorf("update request: %w", err)
61    }
62    return nil
63    }); err != nil {
64        return model.Request{}, fmt.Errorf("with transaction: %w", err)
65    }
66
67    return updatedRequest, nil
68 }

```

Листинг 2.7: Закрытие заявки с проверкой допустимого исходного состояния (close.go)

```

1 package v1
2
3 import (
4     "context"
5     "fmt"
6     "reflect"
7
8     "github.com/cbs-helperbot/helperbot-core/internal/model"
9 )
10
11 func (s *Service) Close(
12     ctx context.Context, requestID model.RequestID,
13 ) (model.Request, error) {
14     var closedRequest model.Request
15
16     if err := s.transactor.WithinTransaction(ctx,
17         func(ctx context.Context) error {

```

```

18     storedRequest, err := s.requestRepository.Get(ctx,
requestID)
19     if err != nil {
20         return fmt.Errorf("get stored request: %w", err)
21     }
22     if reflect.DeepEqual(storedRequest, model.Request{}) {
23         return model.ErrRequestNotFound
24     }
25
26     if storedRequest.Status != model.RequestStatusInProgress &&
27         storedRequest.Status != model.RequestStatusCreated {
28         return model.ErrWrongRequestStatus
29     }
30
31     closedRequest, err = s.requestRepository.Close(ctx,
requestID)
32     if err != nil {
33         return fmt.Errorf("close request: %w", err)
34     }
35     return nil
36 }); err != nil {
37     return model.Request{}, fmt.Errorf("with transaction: %w", err)
38 }
39
40 return closedRequest, nil
41 }

```

Листинг 2.8: Удаление заявки (delete.go)

```

1 package v1
2
3 import (
4     "context"
5     "fmt"
6
7     "github.com/cbs-helperbot/helperbot-core/internal/model"
8 )
9
10 func (s *Service) Delete(
11     ctx context.Context, requestID model.RequestID,
12 ) error {
13     if err := s.transactor.WithinTransaction(ctx,

```

```
14     func(ctx context.Context) error {
15         if err := s.requestRepository.Delete(ctx, requestID); err
16         != nil {
17             return fmt.Errorf("delete request: %w", err)
18         }
19         return nil
20     }); err != nil {
21         return fmt.Errorf("within transaction: %w", err)
22     }
23     return nil
24 }
```


Приложение В. Конфигурация docker-compose инфраструктуры

В приложении приведён файл `docker-compose.yml` из репозитория `helperbot-infrastructure`, описывающий все запускаемые компоненты комплекса: основной кластер PostgreSQL в режиме физической репликации `master-slave`, отдельный кластер для AI-сервиса, контейнер накатывания миграций, серверное ядро *helperbot-core*, Telegram-клиент *helperbot-tg*, внешний AI-сервис *helperbot-ai* и *Ollama-runtime*. Все контейнеры объединены в единую bridge-сеть `helperbot-network`.

Листинг 2.9: Конфигурация docker-compose инфраструктуры Helperbot

```
1 version: '3.8'
2
3 services:
4   postgresql-master:
5     image: bitnamilegacy/postgresql:16
6     container_name: postgresql-master
7     ports:
8       - '5432:5432'
9     volumes:
10      - postgresql_master_data:/bitnami/postgresql
11     environment:
12       - POSTGRESQL_REPLICATION_MODE=master
13       - POSTGRESQL_REPLICATION_USER=repl_user
14       - POSTGRESQL_REPLICATION_PASSWORD=repl_password
15       - POSTGRESQL_USERNAME=postgres
16       - POSTGRESQL_PASSWORD=helperbot
17       - POSTGRESQL_DATABASE=postgres
18       - POSTGRESQL_SYNCHRONOUS_COMMIT=on
19       - POSTGRESQL_NUM_SYNCHRONOUS_REPLICAS=1
20     healthcheck:
21       test: ["CMD-SHELL", "pg_isready -U postgres"]
22       interval: 5s
23       timeout: 5s
24       retries: 5
25     networks:
26       - helperbot-network
27
28   postgresql-slave:
29     image: bitnamilegacy/postgresql:16
30     container_name: postgresql-slave
```

```

31   depends_on:
32     - postgresql-master
33   ports:
34     - '5442:5432'
35   volumes:
36     - postgresql_slave_data:/bitnami/postgresql
37   environment:
38     - ALLOW_EMPTY_PASSWORD=yes
39     - POSTGRESQL_REPLICATION_MODE=slave
40     - POSTGRESQL_REPLICATION_USER=repl_user
41     - POSTGRESQL_REPLICATION_PASSWORD=repl_password
42     - POSTGRESQL_MASTER_HOST=postgresql-master
43     - POSTGRESQL_MASTER_PORT_NUMBER=5432
44   healthcheck:
45     test: ["CMD-SHELL","pg_isready -U postgres"]
46     interval: 5s
47     timeout: 5s
48     retries: 5
49   networks:
50     - helperbot-network
51
52 db-migrations:
53   container_name: db-migrations
54   image: golang:alpine
55   volumes:
56     - ./helperbot-core/migrations:/migrations:ro
57   depends_on:
58     postgresql-master:
59       condition: service_healthy
60   restart: "no"
61   command: >
62     sh -euxc '
63       go install github.com/pressly/goose/v3/cmd/goose@latest &&
64       export PATH="$(go env GOPATH)/bin:$PATH" &&
65       goose -dir /migrations postgres "postgres://postgres:
helperbot@postgresql-master:5432/postgres?sslmode=disable" up
66     '
67   networks:
68     - helperbot-network
69
70 helperbot-core:

```

```
71   build:
72     context: ./helperbot-core
73     dockerfile: dockerfile
74   container_name: helperbot-core
75   volumes:
76     - .env.helperbot-core:/app/.env:ro
77   depends_on:
78     - postgresql-master
79     - postgresql-slave
80   ports:
81     - "8081:8081"
82     - "50051:50051"
83   command: ./main
84   networks:
85     - helperbot-network
86
87 helperbot-tg:
88   build:
89     context: ./helperbot-tg
90     dockerfile: dockerfile
91   container_name: helperbot-tg
92   volumes:
93     - .env.helperbot-tg:/app/.env:ro
94   depends_on:
95     - postgresql-master
96     - helperbot-core
97   command: ./main
98   networks:
99     - helperbot-network
100
101 helperbot-ai:
102   build:
103     context: ./helperbot-ai
104     dockerfile: dockerfile
105   container_name: helperbot-ai
106   volumes:
107     - .env.helperbot-ai:/app/.env:ro
108   depends_on:
109     - postgres-ai-master
110     - postgres-ai-slave
111   ports:
```

```
112     - "8082:8082"
113     - "50052:50052"
114     command: ./main
115     networks:
116     - helperbot-network
117
118 volumes:
119     postgresql_master_data:
120     postgresql_slave_data:
121
122 networks:
123     helperbot-network:
124     driver: bridge
```

В приведённом фрагменте опущены сервисы кластера для базы данных AI-подсистемы (postgres-ai-master, postgres-ai-slave) и контейнер Ollama, поскольку они относятся к внешнему сервису *helperbot-ai* и в настоящей работе не рассматриваются. Контейнер db-migrations запускается однократно после готовности postgresql-master, устанавливает утилиту goose [14] и применяет все накопленные миграции из каталога helperbot-core/migrations.

Приложение Г. Обработчик создания заявки в Telegram-боте

В приложении приведён исходный код обработчика создания заявки библиотекарем из клиентского приложения *helperbot-tg*. Файл расположен по пути `internal/handler/handle_librarian_default_request.go`. Обработчик демонстрирует типичный сценарий взаимодействия Telegram-бота с серверным ядром: библиотекарь выбирает заявку из списка типовых обращений, бот через gRPC вызывает `RequestService.Create` в *helperbot-core*, после чего возвращает пользователю подтверждение.

Листинг 2.10: Обработчик выбора типовой заявки библиотекарем (`handle_librarian_default_request.go`)

```

1 package handler
2
3 import (
4     "context"
5     "fmt"
6     "sort"
7     "strconv"
8     "strings"
9
10    "github.com/cbs-helperbot/helperbot-tg/internal/model"
11    "gopkg.in/telebot.v4"
12 )
13
14 const SelectLibrarianDefaultRequest = "
15     librarian_default_request_selection"
16
17 func (h *Handler) startHandlingLibrarianDefaultRequest(
18     c telebot.Context,
19 ) error {
20     telegramID, err := model.NewTelegramID(c.Chat().ID)
21     if err != nil {
22         return h.processError(c,
23             fmt.Errorf("parse telegram id: %w", err))
24     }
25
26     h.cache.AddMsg(telegramID, c.Message())
27
28     msg, err := h.bot.Send(c.Chat(),

```

```

28     "Выберите вашу заявку из перечисленных ниже "+
29     "или нажмите Другое<>, если не нашли вашу проблему")
30     if err != nil {
31         return h.processError(c,
32             fmt.Errorf("send librarian start message: %w", err))
33     }
34     h.cache.AddMsg(telegramID, msg)
35
36     keys := make([]int, 0, len(model.DefaultRequests))
37     for k := range model.DefaultRequests {
38         keys = append(keys, k)
39     }
40     sort.Ints(keys)
41
42     buttons := make([]inlineBtn, 0, len(keys))
43     for _, k := range keys {
44         req := model.DefaultRequests[k]
45         btn := newInlineBtn(strconv.Itoa(k), req.Description)
46         buttons = append(buttons, btn)
47     }
48
49     defaultRequestMenu := createInlineMenu(
50         SelectLibrarianDefaultRequest, buttons...)
51
52     menuMsg, err := h.bot.Send(c.Chat(),
53         "Выберите заявку:", defaultRequestMenu)
54     if err != nil {
55         return h.processError(c,
56             fmt.Errorf("send librarian request menu: %w", err))
57     }
58     h.cache.AddMsg(telegramID, menuMsg)
59
60     return nil
61 }
62
63 func (h *Handler) processHandleLibrarianDefaultRequest(
64     c telebot.Context,
65 ) error {
66     ctx := context.Background()
67
68     telegramID, err := model.NewTelegramID(c.Chat().ID)

```

```

69     if err != nil {
70         return h.processError(c,
71             fmt.Errorf("parse telegram id: %w", err))
72     }
73
74     data := c.Data()
75     defaultRequestID, err := strconv.Atoi(data)
76     if err != nil {
77         return h.processError(c,
78             fmt.Errorf("invalid request callback data: %w", err))
79     }
80
81     req, ok := model.DefaultRequests[defaultRequestID]
82     if !ok {
83         return h.processError(c,
84             fmt.Errorf("request id not found: %d", defaultRequestID))
85     }
86
87     if strings.Contains(req.Description, "Другое") {
88         return h.startHandlingLibrarianRequestDescription(c)
89     }
90
91     librarian, err := h.userService.GetByTelegramID(ctx, telegramID)
92     if err != nil {
93         return h.processError(c,
94             fmt.Errorf("get librarian: %w", err))
95     }
96
97     if librarian.Role != model.UserRoleLibrarian {
98         return h.processError(c, customMessageError{
99             message: "Это действие может выполнять только библиотекарь",
100             err:      err,
101         })
102     }
103
104     departmentID, ok := librarian.DepartmentID.Get()
105     if !ok {
106         return h.processError(c, customMessageError{
107             message: "У пользователя не указано подразделение",
108             err:      err,

```

```

109     })
110 }
111
112 createdRequest, err := h.requestService.Create(ctx, model.Request{
113     DepartmentID: departmentID,
114     LibrarianID:  librarian.ID,
115     Description:  req.Description,
116     Priority:     req.Priority,
117 })
118 if err != nil {
119     return h.processError(c,
120         fmt.Errorf("create request: %w", err))
121 }
122
123 descRequest, err := h.toDescFromRequestData(ctx, createdRequest)
124 if err != nil {
125     return h.processError(c,
126         fmt.Errorf("convert request: %w", err))
127 }
128
129 h.cache.Delete(telegramID)
130 msg, err := h.bot.Send(c.Chat(),
131     fmt.Sprintf("Созданная вами заявка\n\n%s", descRequest))
132 if err != nil {
133     return h.processError(c,
134         fmt.Errorf("send message: %w", err))
135 }
136 h.cache.AddMsg(telegramID, msg)
137
138 return nil
139 }

```

Метод `startHandlingLibrarianDefaultRequest` вызывается при нажатии библиотекарем кнопки «Создать заявку»: он формирует inline-клавиатуру со списком predefined типовых обращений (`model.DefaultRequests`). После выбора пункта меню Telegram отправляет callback-сообщение, которое обрабатывается методом `processHandleLibrarianDefaultRequest`. Этот метод выполняет три ключевых действия: получает запись пользователя по `telegram_id` (gRPC-вызов `UserService.GetByTelegramID` в ядре), проверяет, что роль пользователя соответствует библиотекарю, и через gRPC-метод `RequestService.Create` передаёт ядру данные новой

заявки. Ядро возвращает заполненную сущность с присвоенным идентификатором и статусом `created`, после чего бот отправляет пользователю подтверждение.

Приложение Д. Снимки экрана интерфейса Telegram-бота

В приложении приведены снимки экрана Telegram-бота *helperbot-tg*, иллюстрирующие отдельные шаги сценариев регистрации сотрудника и работы с заявкой.

На рисунке 2.7 показан второй шаг сценария регистрации сотрудника: бот предлагает новому пользователю выбрать роль из числа допустимых, а в случае выбора роли библиотекаря — указать подразделение ЦБС.

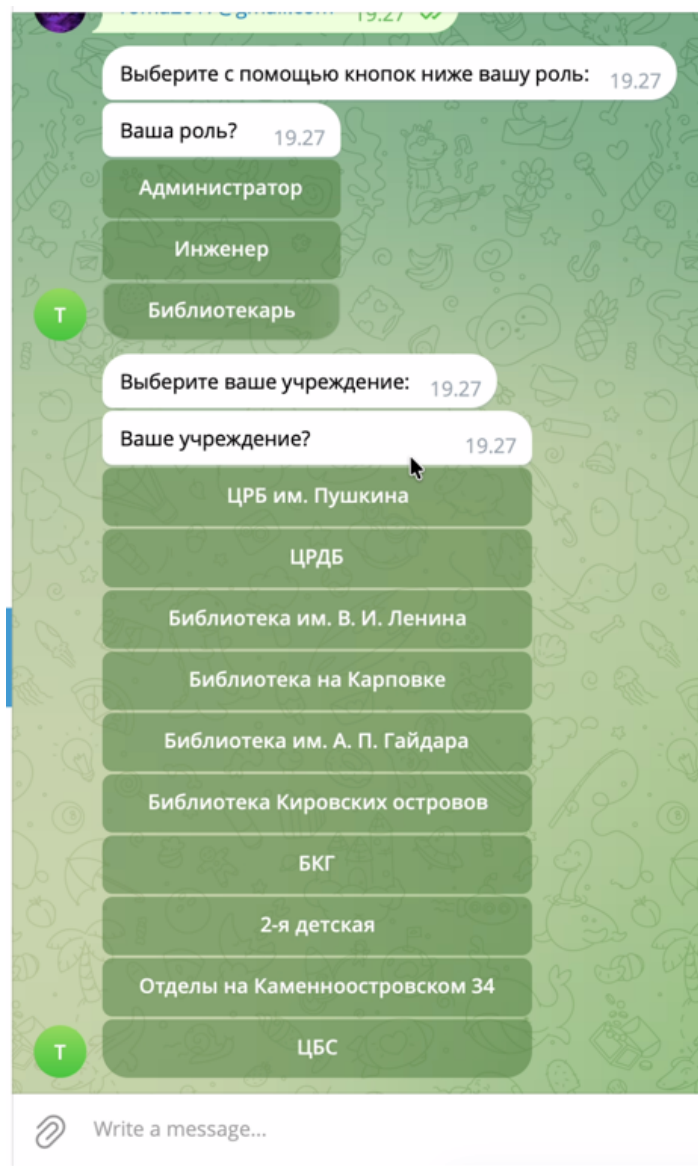


Рисунок 2.7 – Выбор роли и подразделения при регистрации сотрудника

На рисунке 2.8 приведена карточка только что созданной заявки с точки зрения её автора-библиотекаря: номер заявки, подразделение, отсутствующий исполнитель, статус Заявка создана.

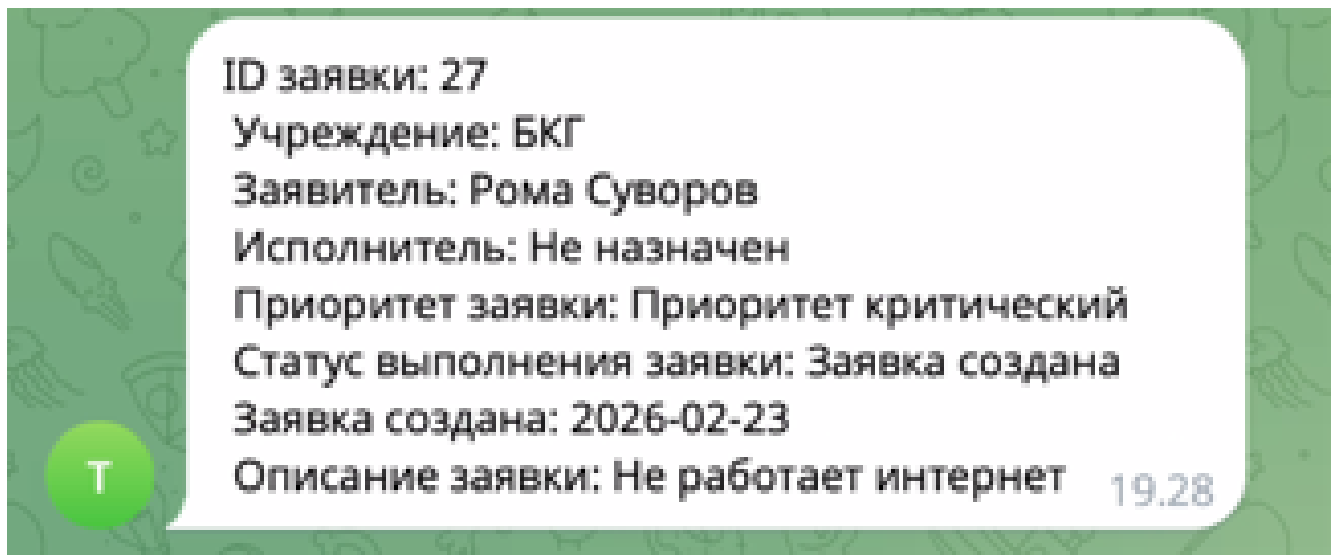


Рисунок 2.8 – Карточка созданной заявки с точки зрения библиотекаря

На рисунке 2.9 показан вид той же заявки после назначения инженера-исполнителя: статус сменился на Заявка в процессе выполнения, под карточкой появились inline-кнопки Отметить как выполненную и Удалить заявку, доступные только назначенному инженеру и администраторам.

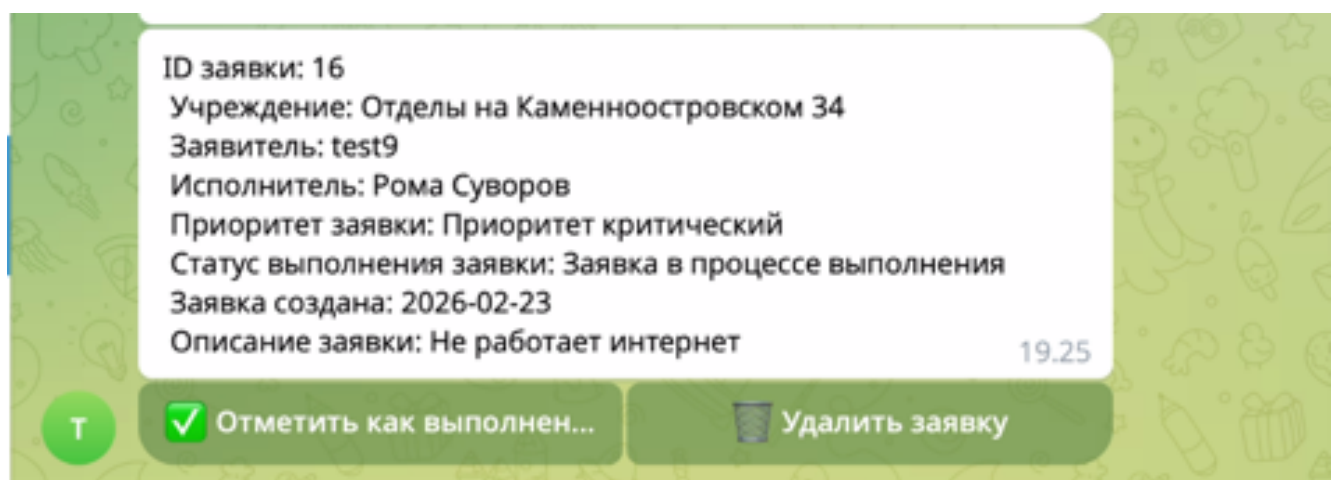


Рисунок 2.9 – Карточка заявки в работе с inline-кнопками смены статуса